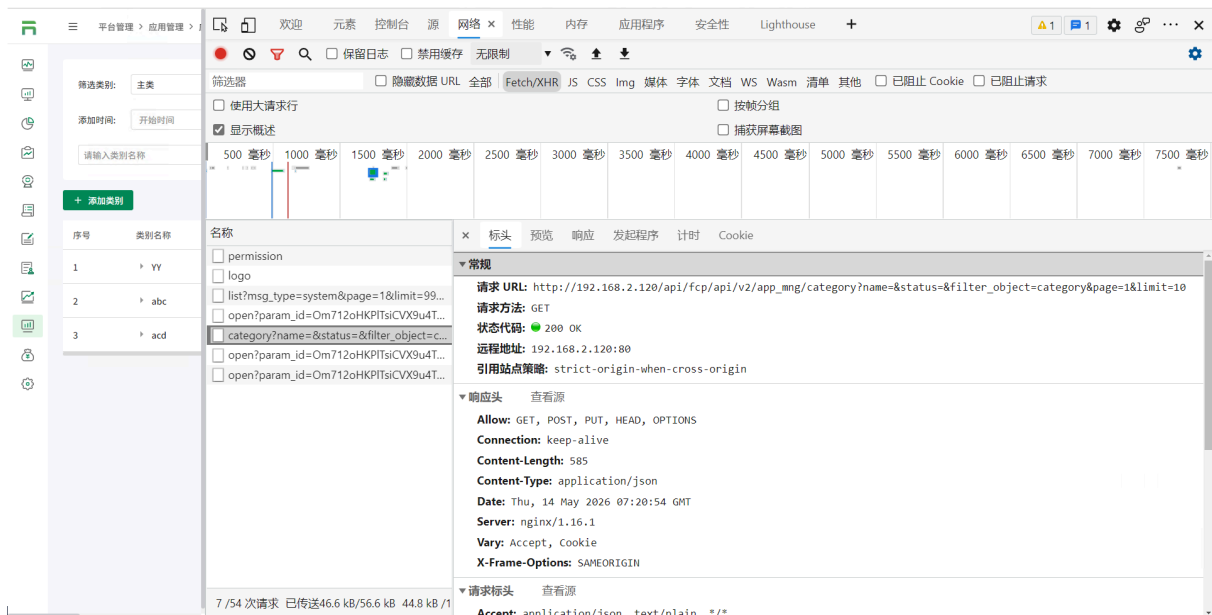


自定义应用相关源码

```
urlpatterns=[
    url(r'^category_simple$', views.CategorySimpleView.as_view()),
    url(r'^category$', views.CategoryView.as_view()),
    url(r'^category_status_mod$', views.CategoryStatusModView.as_view()),
    url(r'^s_category$', views.SCategoryView.as_view()),
    url(r'^move_s_category$', views.MoveSCategoryView.as_view()),
    url(r'^s_category_status_mod$', views.SCategoryStatusModView.as_view()),
    url(r'^check_app_position$', views.CheckAppPositionView.as_view()),
    url(r'^s_category_simple$', views.SCategorySimpleView.as_view()),
    url(r'^unbind_license$', views.UnbindLicenseView.as_view()),
    url(r'^build_in_param$', views.BuildInParameterView.as_view()),
    url(r'^cluster_bind_user$', views.ClusterBindUserView.as_view()),
    url(r'^cluster_simple$', views.ClusterSimpleView.as_view()),
    url(r'^app$', views.AppView.as_view()),
    url(r'^app_base$', views.AppBaseView.as_view()),
    url(r'^app_template$', views.AppTemplateView.as_view()),
    url(r'^list$', views.ListView.as_view()),
    url(r'^app_status$', views.AppStatusView.as_view()),
    url(r'^app_bind_user$', views.AppBindUserView.as_view()),
    url(r'^bind_cluster$', views.BindClusterView.as_view()),
    url(r'^app_license$', views.AppLicenseView.as_view()),
    url(r'^app_license_cluster$', views.AppLicenseClusterView.as_view()),
    url(r'^lic_authorize_user$', views.LicAuthorizeUserView.as_view()),
    url(r'^model_list$', views.ModelListView.as_view()),
    url(r'^model$', views.ModelView.as_view())
]
```

第一类：应用分类



一 应用类别接口category

1. 访问控制与权限

- 权限限制：该接口受到严格保护，仅允许 `admin`（管理员）用户访问。

`views.py`

```

class CategoryView(APIView):
    @method_decorator(permission(usertype=["admin"], permission="plat_set_get"))
    def get(self, request):
        r = Category().get(request)
        return Response(r)

    @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
    def post(self, request):
        r, event = Category().post(request)
        # 记录日志
        if r["code"] == 0:
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                       operate_type="新增应用分类",
                                       operate_content=event, state=True)
        return Response(r)

    @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
    def put(self, request):
        r, event = Category().put(request)
        # 记录日志
        if r["code"] == 0 and event is not None:
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                       operate_type="编辑应用分类",
                                       operate_content=event, state=True)
        return Response(r)

```

GET请求：应用查询

- 操作权限：用户必须拥有 `plat_set_get`（平台设置查询）权限才能成功调用。

2. 关键输入参数

你可以通过 URL 参数来精细化你的查询结果：

```

def get(self, request):
    status = request.GET.get("status", None)
    app_category_id_p = request.GET.get("category_id", None)
    # app_category_id_c = request.GET.get("s_category_id", None)
    filter_object = request.GET.get("filter_object", None)
    category_name = request.GET.get("name", None)

    create_time_s = request.GET.get("s_time", None)
    create_time_e = request.GET.get("e_time", None)

    page_index = request.GET.get("page", 1)
    page_size = request.GET.get("limit", 10)

    result = {
        "code": -1,
        "message": "failed"
    }

    if filter_object not in ["", None, "category", "s_category"]:
        result["err"] = "请指定筛选类别"
        return result

    # 只针对大类分页
    if page_index in ["", None]:
        page_index = 1
    if page_size in ["", None]:
        page_size = 10

    page_index = int(page_index)
    page_size = int(page_size)

```

- **筛选条件：**支持按 `status`（状态）、`name`（名称模糊查询）、以及 `s_time/e_time`（创建时间范围）进行过滤。
- **类别层级：**通过 `filter_object` 参数指定是查询“大类”还是“子类”。
- **分页控制：**使用 `page`（页码）和 `limit`（每页条数，默认 10 条）进行数据分页。

3. 后端处理核心逻辑

代码在执行时遵循以下流程：

- **初始化过滤：**默认只查询外部应用（`is_extern=1`）。


```

... data = {
...     "page": page_index,
...     "limit": page_size,
...     "count": 0,
...     "category": []
... }

... obj = TbAppCategory.objects.filter(is_extern=1)
... if status not in ["", None]:
...     obj = obj.filter(status=status)

... # 当前是否查询子类
... child_flag = False
... if filter_object == "s_category":
...     child_flag = True
...     obj = obj.exclude(pid=0)
... else:
...     obj = obj.filter(pid=0)

... # 当前是否要获取大类下的子类数据
... get_child_flag = False
... if app_category_id_p not in ["", None]:
...     if child_flag:
...         obj = obj.filter(pid=app_category_id_p)
...     else:
...         obj = obj.filter(app_category_id=app_category_id_p)
...     get_child_flag = True

```

```

... if category_name not in ["", None]:
...     obj = obj.filter(category_name__contains=category_name)

... if create_time_s not in ["", None] and create_time_e not in ["", None]:
...     cond = f"create_time BETWEEN '{create_time_s}' AND '{create_time_e}'"
...     obj = obj.extra(where=[cond])

... #
... val = obj.values("app_category_id", "category_name", "order_num",
...                  "intro", "remark", "status", "create_time").order_by("order_num")

... obj_c = obj
... if child_flag: # 筛选的子类 所属的大类
...     pid_list = list(obj.values_list("pid", flat=True))

...     obj = TbAppCategory.objects.filter(is_extern=1, app_category_id__in=pid_list)
...     val = obj.values("app_category_id", "category_name", "order_num",
...                      "intro", "remark", "status", "create_time").order_by("order_num")

```

• 层级识别:

- 如果 `filter_object` 为空或指定为大类，后端会过滤出所有 `pid=0`（父 ID 为 0）的记录。
- 如果查询子类，则会排除 `pid=0` 的记录，或根据指定的 `app_category_id` 查找其关联的子项。

```

...total_num = len(val)

...page_num = int(total_num / page_size)
...else_num = total_num % page_size
...if else_num > 0:
...    page_num = page_num + 1

...if page_index > page_num:
...    page_index = page_num

...d_i_s = page_size * (page_index - 1)
...d_i_e = page_size * page_index

...if d_i_s < 0:
...    d_i_s = 0

...val = val[d_i_s:d_i_e]

```

分页计算

- 自动统计与嵌套：

```

data_cty = []
if not child_flag:
    if not get_child_flag:
        for i in val:
            app_category_id = i["app_category_id"]
            num_c = TbAppCategory.objects.filter(is_extern=1,
                                                    pid=app_category_id).count()
            if TbAppCategory.objects.filter(pid=app_category_id).count() > 0:
                is_exist_s = 1
            else:
                is_exist_s = 0
            d = {
                "app_category_id": app_category_id,
                "category_name": i["category_name"],
                "s_num": num_c,
                "order_num": i["order_num"],
                "intro": i["intro"],
                "remark": i["remark"],
                "status": i["status"],
                "create_time": str(i["create_time"]),
                "s_cate": [],
                "is_exist_s": is_exist_s
            }
            data_cty.append(d)

```

```

else:
    for i in val:
        app_category_id = i["app_category_id"]
        i_obj_c = TbAppCategory.objects.filter(is_extern=1, pid=app_category_id)
        if TbAppCategory.objects.filter(pid=app_category_id).count() > 0:
            is_exist_s = 1
        else:
            is_exist_s = 0
        num_c = i_obj_c.count()
        d = {
            "app_category_id": app_category_id,
            "category_name": i["category_name"],
            "s_num": num_c,
            "order_num": i["order_num"],
            "intro": i["intro"],
            "remark": i["remark"],
            "status": i["status"],
            "create_time": str(i["create_time"]),
            "s_cate": [],
            "is_exist_s": is_exist_s
        }

        i_val_c = i_obj_c.values("app_category_id", "category_name", "order_num",
                                "intro", "remark", "status", "icon_name", "create_time").order_by("order_num")

```

```

            i_val_c = i_obj_c.values("app_category_id", "category_name", "order_num",
                                     "intro", "remark", "status", "icon_name", "create_time").order_by("order_num")

            data_cty_c = []
            for c in i_val_c:
                c_app_category_id = c["app_category_id"]
                icon_name = c["icon_name"]
                num_version = TbAppVersion.objects.filter(is_extern=1,
                                                         category_id=c_app_category_id).count()
                icon_url = f"{PRJ_IMG_APP_PATH}/{c_app_category_id}/{icon_name}"
                c_d = {
                    "s_category_id": c_app_category_id,
                    "s_category_name": c["category_name"],
                    "icon_url": PassWord().encrypt(icon_url),
                    "version_num": num_version,
                    "order_num": c["order_num"],
                    "intro": c["intro"],
                    "remark": c["remark"],
                    "status": c["status"],
                    "create_time": str(c["create_time"])
                }

            data_cty_c.append(c_d)

            d["s_cate"] = data_cty_c
            data_cty.append(d)

```

```

    else:
        for i in val:
            app_category_id = i["app_category_id"]
            num_c = TbAppCategory.objects.filter(is_extern=1,
                                                pid=app_category_id).values().count()
            d = {
                "app_category_id": app_category_id,
                "category_name": i["category_name"],
                "s_num": num_c,
                "order_num": i["order_num"],
                "intro": i["intro"],
                "remark": i["remark"],
                "status": i["status"],
                "create_time": str(i["create_time"]),
                "s_cate": []
            }

            i_obj_c = obj_c.filter(pid=app_category_id)
            i_val_c = i_obj_c.values("app_category_id", "category_name", "order_num",
                                     "intro", "remark", "status", "icon_name", "create_time").order_by("order_num")

```

```

data_cty_c = []
for c in i_val_c:
    c_app_category_id = c["app_category_id"]
    icon_name = c["icon_name"]
    num_version = TbAppVersion.objects.filter(is_extern=1,
                                                category_id=c_app_category_id).count()

    # icon_url = f"{PRJ_IMG_APP_URL_PATH}/{c_app_category_id}/{icon_name}"
    icon_url = f"{PRJ_IMG_APP_PATH}/{c_app_category_id}/{icon_name}"
    c_d = {
        "s_category_id": c_app_category_id,
        "s_category_name": c["category_name"],
        # "icon_url": icon_url,
        "icon_url": Password().encrypt(icon_url),
        "version_num": num_version,
        "order_num": c["order_num"],
        "intro": c["intro"],
        "remark": c["remark"],
        "status": c["status"],
        "create_time": str(c["create_time"])
    }

    data_cty_c.append(c_d)

d["s_cate"] = data_cty_c
data_cty.append(d)

data["category"] = data_cty
data["count"] = total_num

```

```

result={
    "code":0,
    "message":"success",
    "data":data
}
return result

```

- **子类计数**：在查询大类时，代码会自动统计每个大类下包含的子类别数量（`s_num`）。
- **嵌套数据**：它会进一步查询每个大类下的具体子类详情（`s_cate` 列表），并统计每个子类关联的版本数量。
- **安全处理**：为了安全起见，所有返回的图标 URL（`icon_url`）都会经过 `Password().encrypt` 进行加密处理，而不是直接暴露原始路径。

4. 响应数据结构

接口最终会返回一个标准化的 JSON 对象：

- **状态码（`code`）**：`0` 表示成功。
- **分页信息**：包含当前页码、每页限制和符合条件的总条数（`count`）。

- **核心数据 (category)**: 一个包含所有类别详情的数组，每个元素内可能嵌套了子类别列表。

POST请求：新增应用分类

这是当你点击页面上的“添加类别”按钮并提交表单时触发的逻辑。

- **操作权限**: 用户必须拥有 `plat_set_op`（平台操作查询）权限才能成功调用。
- **核心逻辑流程**:

```
def post(self, request):
    category_name = request.POST.get("category_name", None)
    order_num = request.POST.get("order_num", None)
    remark = request.POST.get("remark", None)

    result = {
        "code": -1,
        "message": "failed"
    }
    event = None

    if category_name in ["", None]:
        result["err"] = "请指定类别名称"
        return result, event

    if order_num in ["", None]:
        result["err"] = "请指定排序"
        return result, event
```

1. **必填校验**: 检查分类名称 (`category_name`) 和排序 (`order_num`) 是否为空。

```
obj = TbAppCategory.objects.filter(category_name=category_name)
val = obj.values()
if len(val) > 0:
    result["err"] = "存在同名类别"
    return result, event
```

1. **重名检查**: 在数据库中查找是否已有同名的分类。如果存在，直接返回“存在同名类别”错误。

```

ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
data = {
    "category_name": category_name,
    "order_num": order_num,
    "remark": remark,
    "create_time": ctime,
    "update_time": ctime
}

TbAppCategory.objects.create(**data)

result = {
    "code": 0,
    "message": "success"
}

# event = f"分类名称: {category_name}"
event = '''<p class="log_log_obj"><span class="obj">分类名称: </span><span class="val">%s</span></p>''' % category_name

return result, event

```

1. **入库**：创建新记录，并自动记录 `create_time` 和 `update_time`。
2. **生成日志**：构造一段 HTML 格式的文本（例如：`分类名称: 某某`），交给外层函数写入系统审计日志。

PUT：编辑应用分类

这是当你修改现有分类信息（如改名、改排序、改备注）时触发的逻辑。

- **操作权限**：用户必须拥有 `plat_set_op`（平台操作查询）权限才能成功调用。
- **核心逻辑流程**：
 1. **变更检测（关键点）**：


```

def put(self, request):
    app_category_id = request.POST.get("category_id", None)
    category_name = request.POST.get("category_name", None)
    order_num = request.POST.get("order_num", None)
    remark = request.POST.get("remark", None)

    result = {
        "code": -1,
        "message": "failed"
    }
    event = None

    if app_category_id in ["", None]:
        result["err"] = "请指定类别id"
        return result, event
    if category_name in ["", None]:
        result["err"] = "请指定类别名称"
        return result, event
    if order_num in ["", None]:
        result["err"] = "请指定排序"
        return result, event

    obj = TbAppCategory.objects.filter(is_extern=1, pid=0, app_category_id=app_category_id)
    val = obj.values()
    if len(val) == 0:
        result["err"] = "类别不存在"
        return result, event

    category_name_ori = val[0]["category_name"]
    order_num_ori = val[0]["order_num"]
    remark_ori = val[0]["remark"]

```

```

        cg_flag_dict = {
            "category_name": False,
            "order_num": False,
            "remark": False
        }

        if category_name != category_name_ori:
            s_obj = TbAppCategory.objects.filter(category_name=category_name)
            if s_obj.count() > 0:
                result["err"] = "存在同名类别"
                return result, event

            cg_flag_dict["category_name"] = True
            if order_num != order_num_ori:
                cg_flag_dict["order_num"] = True
            if remark != remark_ori:
                cg_flag_dict["remark"] = True

        # 无修改
        if True not in cg_flag_dict.values():
            result = {
                "code": 0,
                "message": "success"
            }
            return result, event

```

- 代码会对比前端传来的新数据与数据库里的旧数据。
 - 它使用了一个 `cg_flag_dict` 字典来标记哪些字段变了（名称、排序、备注）。
 - **优化逻辑**：如果用户什么都没改就点了保存，接口会直接返回成功，不会操作数据库。
1. **重名校验**：只有当用户**修改了名称**时，才会去检查新名称是否与现有的其他类别冲突。

```

#
ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
data = {
    "category_name": category_name,
    "order_num": order_num,
    "remark": remark,
    "update_time": ctime
}

obj.update(**data)

result = {
    "code": 0,
    "message": "success"
}

```

1. 局部更新：只更新发生变化的字段，并刷新 `update_time`。

```

# 日志
from common.config.project import LOG_PREFIX

# log_bef = "%s类别名称: %s; 排序: %s" % (LOG_PREFIX["bef"], category_name_ori, val[0]["order_num"])
# log_aft = "%s类别名称: %s; 排序: %s" % (LOG_PREFIX["aft"], category_name, order_num)
#
# event = "%s\n%s" % (log_bef, log_aft)
#
log_bef = '''<p class="log_log_content bef"><span class="bef"><span><span class="obj">类别名称: </span><span class="val"></span></span><span class="obj">排序: </span><span class="val"></span></span></p>''' \
    % (LOG_PREFIX["bef"], category_name_ori, order_num_ori)
category_name_css = ""
order_num_css = ""
if cg_flag_dict["category_name"]:
    category_name_css = "changed_val"
if cg_flag_dict["order_num"]:
    order_num_css = "changed_val"
log_aft = '''<p class="log_log_content aft"><span class="aft"><span><span class="obj">类别名称: </span><span class="val"></span></span><span class="obj">排序: </span><span class="val"></span></span></p>''' \
    % (LOG_PREFIX["aft"], category_name_css, category_name, order_num_css, order_num)
event = "%s\n%s" % (log_bef, log_aft)

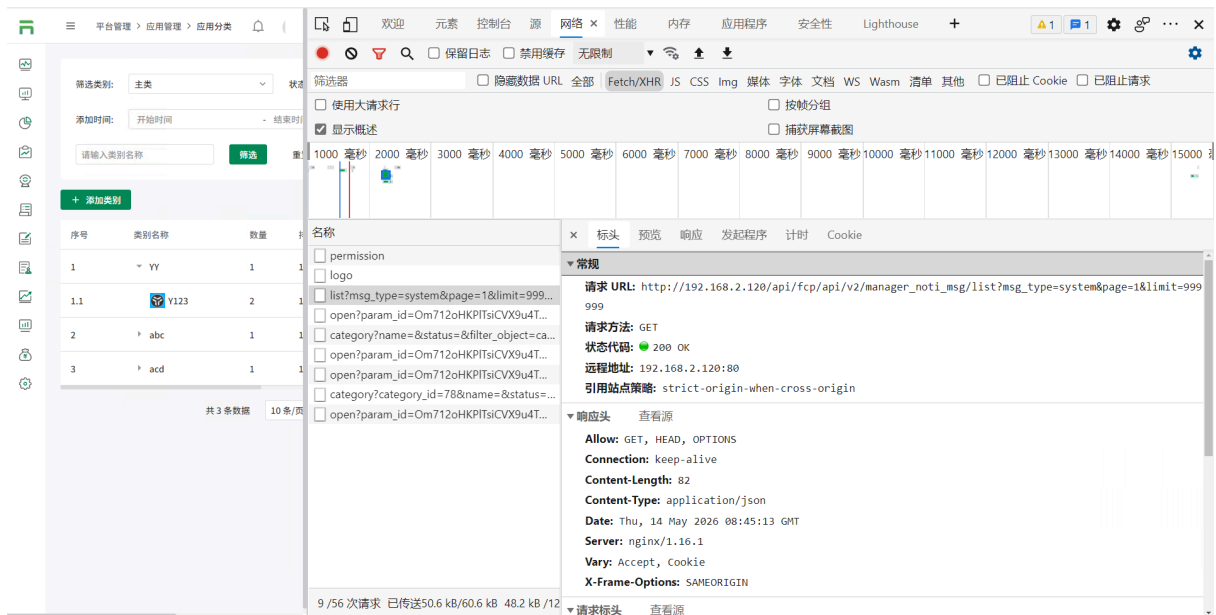
return result, event

```

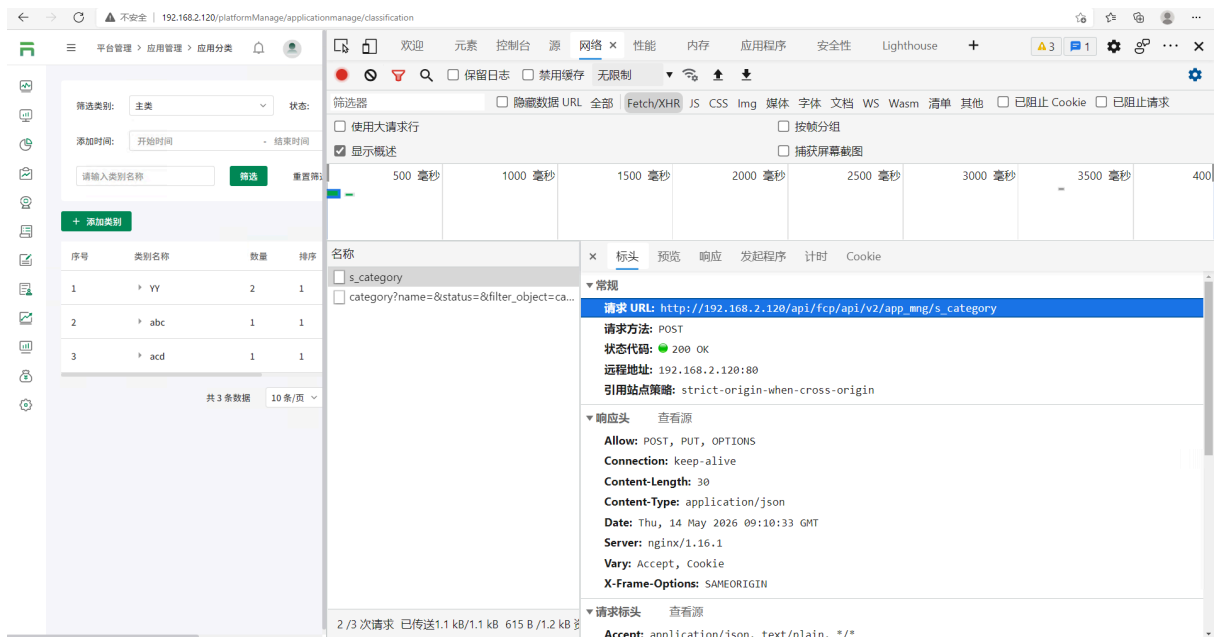
1. 详细审计日志:

- 这是该代码的一个特色。它会对比“修改前”和“修改后”的数据，并生成带有 HTML 标签（如 `<span>`）的日志。
- 在系统日志里，它能高亮显示用户具体改了哪个值（比如从“类别A”变成了“类别B”）。

二 获取“管理员通知消息列表”(关系不大)



三 添加子应用（在分类下添加具体应用） s_category



views.py

```

class SCategoryView(APIView):
    @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
    def post(self, request):
        r, event = CategoryApp().post(request)
        # 记录日志
        if r["code"] == 0:
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                         operate_type="新增应用",
                                         operate_content=event, state=True)
        return Response(r)

    @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
    def put(self, request):
        r, event = CategoryApp().put(request)
        # 记录日志
        if r["code"] == 0 and event is not None:
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                         operate_type="编辑应用",
                                         operate_content=event, state=True)
        return Response(r)

```

POST：新增子应用

后端核心处理逻辑

第一步：严格的预校验

```

def post(self, request):
    pid = request.POST.get("category_id", None)
    file_app_icon = request.FILES.getlist("app_icon")

    category_name = request.POST.get("app_name", None)
    order_num = request.POST.get("order_num", None)
    intro = request.POST.get("intro", None)

    result = {
        "code": -1,
        "message": "failed"
    }
    event = None

    if len(file_app_icon) == 0:
        result["err"] = "请上传应用图标文件"
        return result, event

    if category_name in ["", None]:
        result["err"] = "请指定应用名称"
        return result, event

    if order_num in ["", None]:
        result["err"] = "请指定排序"
        return result, event

    obj = TbAppCategory.objects.filter(category_name=category_name)
    val = obj.values()
    if len(val) > 0:
        result["err"] = "存在同名类别"
        return result, event

```

- **文件校验**：必须上传图标文件，否则报错“请上传应用图标文件”。
- **必填项校验**：名称和排序不能为空。
- **重名检查**：在整个 `TbAppCategory` 表中检查是否有同名的应用/类别，防止名称冲突。

第二步：数据库记录创建

```

app_icon_file = file_app_icon[0]
icon_name = app_icon_file.name

ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
data = {
    "category_name": category_name,
    "order_num": order_num,
    "intro": intro,
    "icon_name": icon_name,
    "pid": pid,
    "create_time": ctime,
    "update_time": ctime
}

app_id = TbAppCategory.objects.create(**data).app_category_id

```

- 系统将应用信息存入 `TbAppCategory` 表。
- **注意：**这里后端采用了同一张表存储“大类”和“小应用”的设计模式。通过 `pid`（父 ID）来区分：`pid=0` 通常是大类，`pid > 0` 的记录则是属于某个大类的具体应用。

第三步：文件系统操作（图标保存）

这是该接口比之前复杂的地方，涉及到了服务器磁盘操作：

```

... if not os.path.isdir(self.img_app_path):
...     # 创建目录
...     os.mkdir(self.img_app_path)

... app_icon_path = os.path.join(self.img_app_path, str(app_id))
... if not os.path.isdir(app_icon_path):
...     # 创建目录
...     os.mkdir(app_icon_path)

... file_path = os.path.join(app_icon_path, icon_name)

... fp = open(file_path, "wb")
... for f_data in app_icon_file.chunks():
...     fp.write(f_data)
... fp.close()

```

1. **创建目录**：代码会检查预设的图片存储根目录是否存在，不存在则创建。
2. **ID 隔离存储**：以新生成的 `app_id` 为名创建一个独立的文件夹。这样做是为了防止不同应用的图标文件名相同导致覆盖。
3. **二进制写入**：通过 `app_icon_file.chunks()` 分块读取上传的文件并写入磁盘。这是一种处理文件上传的内存友好方式，可以防止大文件直接撑爆内存。

第四步：审计日志记录

```

result = {
    "code": 0,
    "message": "success"
}

# event = f"应用名称: {category_name}"
event = '''<p class="log log_obj"><span class="obj">应用名称: </span><span class="val">%s</span></p>''' % category_name

return result, event

```

- 接口执行成功后，外层 `SCategoryView` 会触发日志记录。
- 日志类型为“新增应用”。
- 日志内容会自动带上新增加的应用名称，方便管理员后续追溯。

PUT：编辑子应用

1. 数据比对 (`cg_flag_dict`)

```
def put(self, request):
    app_category_id = request.POST.get("s_category_id", None)
    file_app_icon = request.FILES.getlist("app_icon")

    category_name = request.POST.get("app_name", None)
    order_num = request.POST.get("order_num", None)
    intro = request.POST.get("intro", None)

    result = {
        "code": -1,
        "message": "failed"
    }
    event = None

    if app_category_id in ["", None]:
        result["err"] = "请指定应用id"
        return result, event
    if category_name in ["", None]:
        result["err"] = "请指定应用名称"
        return result, event
    if order_num in ["", None]:
        result["err"] = "请指定排序"
        return result, event

    obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=app_category_id).exclude(pid=0)
    val = obj.values()
    if len(val) == 0:
        result["err"] = "类别不存在"
        return result, event

    category_name_ori = val[0]["category_name"]
    order_num_ori = val[0]["order_num"]
    intro_ori = val[0]["intro"]
```

代码并没有一上来就更新数据库，而是先做了一个“查重”和“比对”：

```

cg_flag_dict = {
    "category_name": False,
    "order_num": False,
    "intro": False,
    "icon_name": False
}

if category_name != category_name_ori:
    s_obj = TbAppCategory.objects.filter(category_name=category_name)
    if s_obj.count() > 0:
        result["err"] = "存在同名类别"
        return result, event

    cg_flag_dict["category_name"] = True
if str(order_num) != str(order_num_ori):
    cg_flag_dict["order_num"] = True
if intro != intro_ori:
    cg_flag_dict["intro"] = True

if len(file_app_icon) > 0:
    cg_flag_dict["icon_name"] = True

# 无修改
if True not in cg_flag_dict.values():
    result = {
        "code": 0,
        "message": "success"
    }
    return result, event

```

- **变动追踪**：它定义了一个 `cg_flag_dict`，专门记录哪些字段（名称、排序、简介、图标）被改动了。
- **静默返回**：如果你点开编辑却什么都没改就点了保存，代码会通过 `if True not in cg_flag_dict.values()` 判断出没有任何修改，直接返回成功。这样可以有效减少数据库和磁盘的压力。

```
...ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
...data = {
...     "category_name": category_name,
...     "order_num": order_num,
...     "intro": intro,
...     "update_time": ctime
...}
```

2. 严谨的图标更新与“大扫除”

这是最核心的逻辑：

```

.....if len(file_app_icon) > 0:
.....    app_icon_file = file_app_icon[0]
.....    icon_name = app_icon_file.name

.....    data["icon_name"] = icon_name

.....    if not os.path.isdir(self.img_app_path):
.....        # 创建目录
.....        os.mkdir(self.img_app_path)

.....    app_icon_path = os.path.join(self.img_app_path, str(app_category_id))
.....    if not os.path.isdir(app_icon_path):
.....        # 创建目录
.....        os.mkdir(app_icon_path)
.....        f_list = []
.....    else:
.....        f_list = os.listdir(app_icon_path)

.....    file_path = os.path.join(app_icon_path, icon_name)

.....    fp = open(file_path, "wb")
.....    for f_data in app_icon_file.chunks():
.....        fp.write(f_data)
.....    fp.close()

.....    for f in f_list:
.....        af = os.path.join(app_icon_path, f)
.....        os.remove(af)

.....    obj.update(**data)

.....    result = {
.....        "code": 0,
.....        "message": "success"
.....    }

```

- **按需更新：**只有当你上传了新的 `file_app_icon` 时，才会触发文件处理逻辑。
- **旧文件清理（关键！）：**在保存新图标之前，代码会先执行 `os.listdir` 获取该应用文件夹下的所有旧文件，并在新文件写入后，通过一个循环 `os.remove(af)` 把旧图标全部删掉。

为什么要这么做？ 这样可以保证服务器磁盘上每个应用永远只有一个最新的图标，不会因为反复修改而堆积大量无用的垃圾文件。

3. 自动生成“对比式”审计日志

最后一部分展示了系统是如何生成日志的：

```

# 日志
from common.config.project import LOG_PREFIX

# icon_url = f"{self.img_app_url_path}/{app_category_id}/{icon_name}"

#
# log_bef = "%s应用图标: %s; 应用名称: %s; 排序: %s" % (LOG_PREFIX["bef"], icon_url, val[0]["category_name"], val[0]["order_num"])
# log_aft = "%s应用图标: %s; 应用名称: %s; 排序: %s" % (LOG_PREFIX["aft"], icon_url, category_name, order_num)

# log_bef = "%s应用名称: %s; 排序: %s" % (LOG_PREFIX["bef"], category_name_ori, val[0]["order_num"])
# log_aft = "%s应用名称: %s; 排序: %s" % (LOG_PREFIX["aft"], category_name, order_num)
#
# event = "%s\n%s" % (log_bef, log_aft)

log_bef = '''<p class=log_log_content bef><span class=bef>%s</span><span class=obj>应用名称: </span><span class=val>%s</span><span class=obj>排序: </span><span class=val>%s</span></p>''' \
    % (LOG_PREFIX["bef"], category_name_ori, order_num_ori)

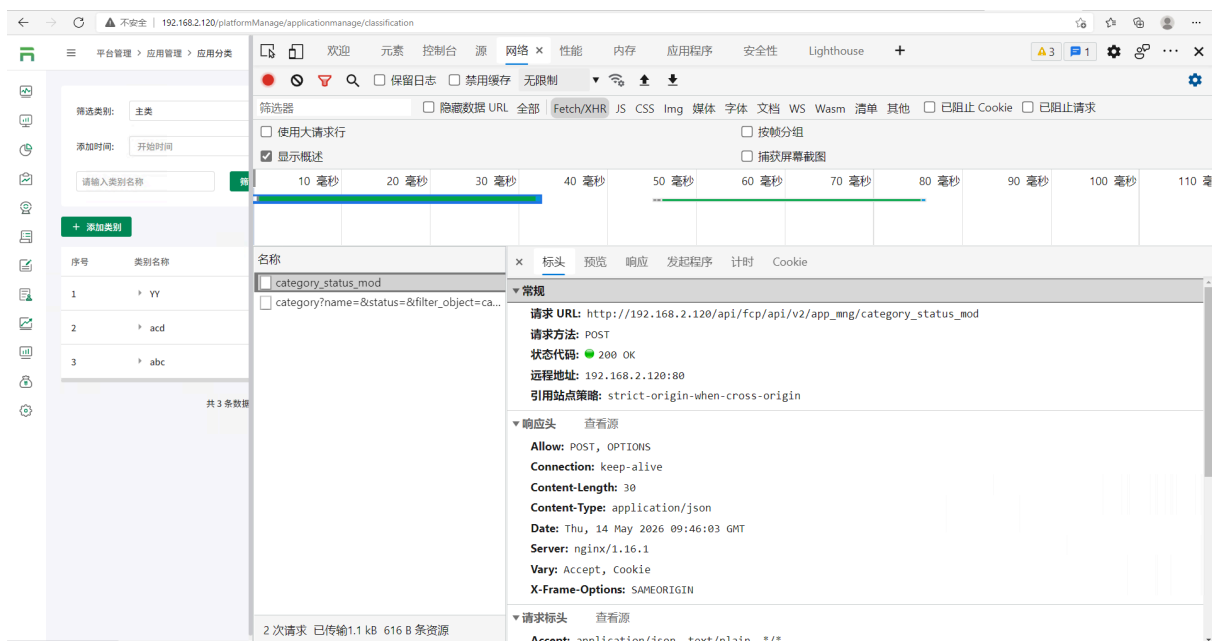
category_name_css = ""
order_num_css = ""
if cg_flag_dict["category_name"]:
    category_name_css = "changed_val"
if cg_flag_dict["order_num"]:
    order_num_css = "changed_val"
log_aft = '''<p class=log_log_content aft><span class=aft>%s</span><span class=obj>应用名称: </span><span class=val>%s</span><span class=obj>排序: </span><span class=val>%s</span></p>''' \
    % (LOG_PREFIX["aft"], category_name_css, category_name, order_num_css, order_num)
event = "%s%s" % (log_bef, log_aft)

return result, event

```

- **HTML 差异化显示：**系统会构造两个字符串：`log_bef`（修改前）和 `log_aft`（修改后）。
- **高亮变化：**如果某个字段变了，它会给对应的 CSS 类加上 `changed_val`。
- **最终效果：**管理员在后台看到的日志不是简单的“修改了应用”，而是像“应用名称：旧名字 → 新名字；排序：1 → 2”这样一目了然的对比。

四 发布和下架应用类别category_status_mod POST



views.py

```

class CategoryStatusModView(APIView):
    @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
    def post(self, request):
        r, event = CategoryStatusMod().post(request)
        # 记录日志
        if r["code"] == 0:
            op_type = event.split(" --- ")[0]
            op_type = f"{op_type}应用分类"
            event = event.split(" -- ")[1]
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                       operate_type=op_type,
                                       operate_content=event, state=True)
        return Response(r)

```

1. 接口功能：主分类状态的“总开关”

这个接口是用来发布或下架**整个大类**（如“办公插件”这个大类）的。因为它处于层级结构的顶端，所以它的变动会直接影响到下面所有的小应用。

```

class CategoryStatusMod(object):
    def post(self, request):
        category_id = request.POST.get("category_id", None)
        status = request.POST.get("status", None)

        result = {
            "code": -1,
            "message": "failed"
        }
        event = None
        op_type = None

        if category_id in ["", None]:
            result["err"] = "请指定类别"
            return result, event
        if status in ["", None] and status not in [0, 1, "0", "1"]:
            result["err"] = "请确定发布/下架"
            return result, event

        obj = TbAppCategory.objects.filter(is_extern=1, pid=0,
                                           app_category_id=category_id).exclude(status=status)
        val = obj.values("app_category_id", "category_name")
        if len(val) == 0:
            result["err"] = "类别不存在"
            return result, event

```

```

...     ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
...     data = {
...         "status": status,
...         "update_time": ctime
...     }

```

2. 关键代码逻辑实现

```

if status in ["0", 0]:
    op_type = "下架"

    pid = category_id
    child_obj = TbAppCategory.objects.filter(is_extern=1, pid=pid).exclude(status=0)
    ct_id_list = child_obj.values_list("app_category_id")
    app_ver_obj = TbAppVersion.objects.filter(category_id__in=ct_id_list).exclude(status=0)

    app_ver_obj.update(**data)
    child_obj.update(**data)

else:
    op_type = "发布"

obj.update(**data)

result = {
    "code": 0,
    "message": "success"
}

```

A. 下架逻辑（最核心的区别）

当 `status` 为 `0`（下架）时，代码执行了一套“连坐”逻辑：

1. **锁定子应用**：它通过 `pid = category_id` 找到了这个 `Appcategory` 大类下所有的子应用（`child_obj`）。
2. **锁定版本**：接着，它拿到了所有子应用的 ID 列表（`ct_id_list`），并找到了这些子应用对应的所有版本记录（`app_ver_obj`）。
3. **一键全关**：
 - 更新所有版本记录的状态为下架。
 - 更新所有子应用的状态为下架。
 - 更新大类本身的状态为下架（通过 `else` 块外或后续逻辑完成）。

为什么要这么做？ 这是一个安全机制。如果大类都下架了，里面的子应用和版本必须全部强制隐藏，防止用户通过非法路径访问到“孤儿”应用。

B. 发布逻辑

当 `status` 为 `1`（发布）时，逻辑相对简单：

- 它只执行 `obj.update(data)`，即仅开启大类本身。
- **注意：**它并不会自动把下面的子应用也全部发布。这给了管理员留了余地——大类开了，但里面的应用可以一个个手动检查后再发布。

C. 视图层的日志解析

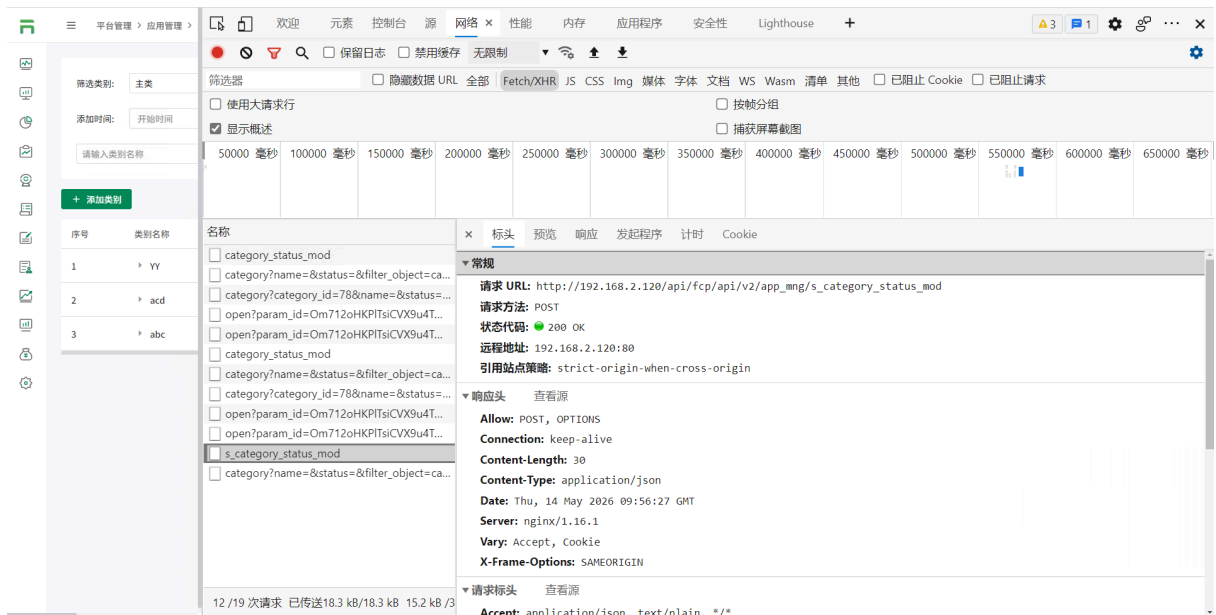
```
category_name = val[0]["category_name"]
# event = f"{op_type} -- 分类名称: {category_name}"
event = '''<p class="log_log_obj"><span class="obj">分类名称: </span><span class="val">%s</span></p>''' \
        % category_name
event = "%s -- %s" % (op_type, event)

return result, event
```

视图函数用了同样的 `split(" -- ")` 技巧：

- 如果逻辑层返回 `"下架 -- <p>...</p>"`，视图层会把它拼成 `operate_type="下架应用分类"`。
- 这种设计保证了日志系统能够准确区分“大类操作”和“子应用操作”。

五 发布和下架子应用s_category_status_mod POST



views.py

```
class SCategoryStatusModView(APIView):
    @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
    def post(self, request):
        r, event = CategoryAppStatusMod().post(request)
        # 记录日志
        if r["code"] == 0:
            op_type = event.split(" -- ")[0]
            op_type = f"{op_type}应用"
            event = event.split(" -- ")[1]
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                       operate_type=op_type,
                                       operate_content=event, state=True)
        return Response(r)
```

1. 接口基本信息

- 路由: `/s_category_status_mod`
- 核心逻辑类: `CategoryAppStatusMod`
- 作用: 改变某个具体应用的可见性, 并自动处理与其关联的“版本”或“父分类”状态。

```

class CategoryAppStatusMod(object):
    def post(self, request):
        category_id = request.POST.get("s_category_id", None)
        status = request.POST.get("status", None)

        result = {
            "code": -1,
            "message": "failed"
        }
        event = None
        op_type = None

        if category_id in ["", None]:
            result["err"] = "请指定应用"
            return result, event
        if status in ["", None] and status not in [0, 1, "0", "1"]:
            result["err"] = "请确定发布/下架"
            return result, event

        obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)\
            .exclude(pid=0).exclude(status=status)
        val = obj.values("app_category_id", "category_name", "pid")
        if len(val) == 0:
            result["err"] = "应用不存在"
            return result, event

        ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        data = {
            "status": status,
            "update_time": ctime
        }

```

2. 核心处理逻辑（联动机制）

这是该接口最聪明的地方，它根据你是要“下架”还是“发布”，执行了两套完全不同的联动逻辑：

```

if status in ["0", 0]:
    op_type = "下架"

    app_ver_obj = TbAppVersion.objects.filter(is_extern=1, category_id=category_id).exclude(status=0)
    app_ver_obj.update(**data)

```

A. 下架逻辑（向下联动：应用 → 版本）

当 `status` 为 `0`（下架）时：

- **逻辑**：系统认为如果应用本身都不见了，那么它旗下的所有版本也不应该再被使用。
- **操作**：代码会去 `TbAppVersion` 表里，把所有属于该应用的**版本记录**也一并修改为“下架”状态。

- **目的：**确保清理彻底，防止应用下架后，其旧版本的安装包还能通过某些路径被查到。

```

else:
    op_type = "发布"

    pid = val[0]["pid"]
    p_obj = TbAppCategory.objects.filter(is_extern=1, pid=0, status=0, app_category_id=pid)
    p_obj.update(**data)

obj.update(**data)

```

B. 发布逻辑（向上联动：应用 → 父分类）

当 `status` 为 `1`（发布）时（见 `else` 块）：

- **逻辑：**如果一个子应用要发布，但它所属的“大类”还是下架状态，用户依然看不到它。
- **操作：**系统会自动找到该应用的父级分类（`pid`），如果父分类是下架状态，**系统会自动把父分类也给发布了。**
- **目的：**保证路径畅通。只要你发布了一个应用，系统就确保它的“大门”（父分类）是敞开的。

```

result = {
    "code": 0,
    "message": "success"
}

category_name = val[0]["category_name"]
# event = f"{op_type} -- 应用名称: {category_name}"
event = '''<p class="log_log_obj"><span class="obj">应用名称: </span><span class="val">%s</span></p>''' \
    % category_name
event = "%s -- %s" % (op_type, event)

return result, event

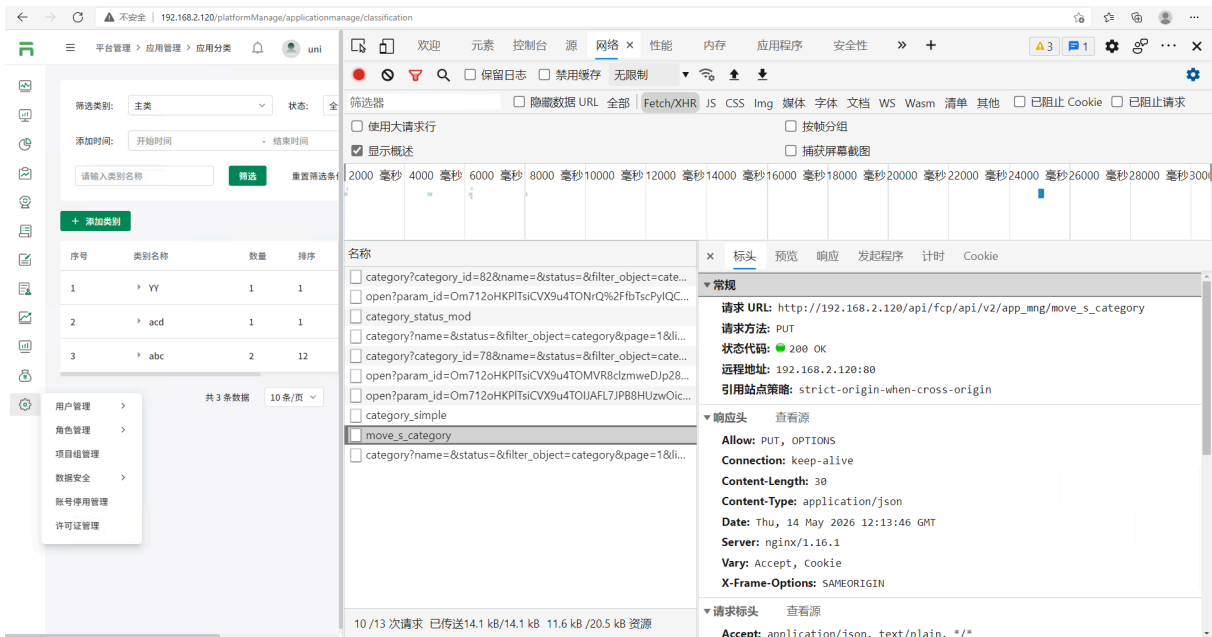
```

3. 日志记录逻辑

视图函数（`SCategoryStatusModView`）延续了项目统一的风格：

- **动态解析：**从逻辑层拿回带有 `-` 分隔符的字符串。
- **精准分类：**解析出操作类型是“发布”还是“下架”，并在日志中拼凑出类似“**发布应用**”或“**下架应用**”的字样。
- **详细内容：**记录具体是哪个应用被操作了。

六 移动子应用move_s_category PUT



views.py

```
class MoveSCategoryView(APIView):  
    @method_decorator(permission(user_type=["admin"], permission="plat_set_op"))  
    def put(self, request):  
        r = CategoryApp().put_move(request)  
        return Response(r)
```

1. 接口基本定义

- 路由: `move_s_category`。
- 视图类: `MoveSCategoryView`，仅允许具有 `plat_set_op` 权限的管理员通过 `PUT` 请求调用。
- 核心逻辑函数: `CategoryApp().put_move(request)`。

2. 关键输入参数

```

def put_move(self, request):
    app_category_id = request.POST.get("s_category_id", None)
    new_pid = request.POST.get("new_category_id", None)
    new_order_num = request.POST.get("order_num", None)

    result = {
        "code": -1,
        "message": "failed"
    }
    if app_category_id in ["", None]:
        result["err"] = "请指定应用id"
        return result
    if new_pid in ["", None]:
        result["err"] = "请指定类别id"
        return result
    if new_order_num in ["", None]:
        result["err"] = "请指定排序"
        return result

    obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=app_category_id).exclude(pid=0)
    val = obj.values("pid", "order_num", "status")

```

接口接收三个核心参数：

1. `s_category_id`：需要移动的目标应用 ID。
2. `new_category_id`：目标新大类的 ID（即新的 `pid`）。
3. `order_num`：应用在目标分类下的新排序位置。

3. 后端处理逻辑与安全机制

```

if len(val) > 0:
    pid = val[0]["pid"]
    order_num = val[0]["order_num"]
    status = val[0]["status"]

    data = {}
    if str(new_order_num) != str(order_num):
        data["order_num"] = new_order_num

    new_p_status = None
    if str(new_pid) != str(pid):
        n_p_obj = TbAppCategory.objects.filter(is_extern=1, pid=0, app_category_id=new_pid)
        if n_p_obj.count() > 0:
            data["pid"] = new_pid

            new_p_status = n_p_obj.values_list("status", flat=True)[0]
        else:
            result["err"] = "类别不存在"
            return result

```

该接口的实现不仅是简单的父 ID 更新，还包含了复杂的状态级联处理：

```

... if len(data) > 0:
...     ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
...     data["update_time"] = ctime

...     from django.db import transaction
...     with transaction.atomic():
...         if new_p_status == 0:
...             # 被移动的应用子类如果未被下架的话下架，应用子类下的版本也下架
...             if status != 0:
...                 data["status"] = 0

...             # 应用子类下的已发布的版本
...             from dbtable.app import AppVersion as TbAppVersion
...             v_obj = TbAppVersion.objects.filter(category_id=app_category_id, status=1)
...             v_obj.update(status=new_p_status, update_time=ctime)

...         obj.update(**data)

...     result = {
...         "code": 0,
...         "message": "success"
...     }

... else:
...     result["err"] = "应用不存在"

... return result

```

A. 差异化更新

代码会对比 `new_pid` 和 `new_order_num` 是否真的发生了改变。只有当检测到参数变动时，才会记录 `update_time` 并触发数据库写操作，这种设计能有效减少不必要的性能损耗。

B. 严格的事务保护 (`transaction.atomic`)

代码引入了 Django 的数据库事务处理。这意味着更新应用信息、更新所属版本状态等一系列操作要么全部成功，要么在出错时全部回滚，防止出现“应用移过去了，但版本数据对不上”的脏数据情况。

C. 状态强制对齐（核心逻辑）

这是该接口最关键的业务规则：

- **检测目标大类状态**：代码会查询新大类（新父节点）的当前状态 (`new_p_status`)。
- **向下兼容下架**：如果目标大类目前是下架状态 (`status=0`)，那么：
 - 被移动的应用也会被强制设为下架。
 - 该应用下所有原本处于“已发布”状态的版本，也会被同步修改为下架状态。

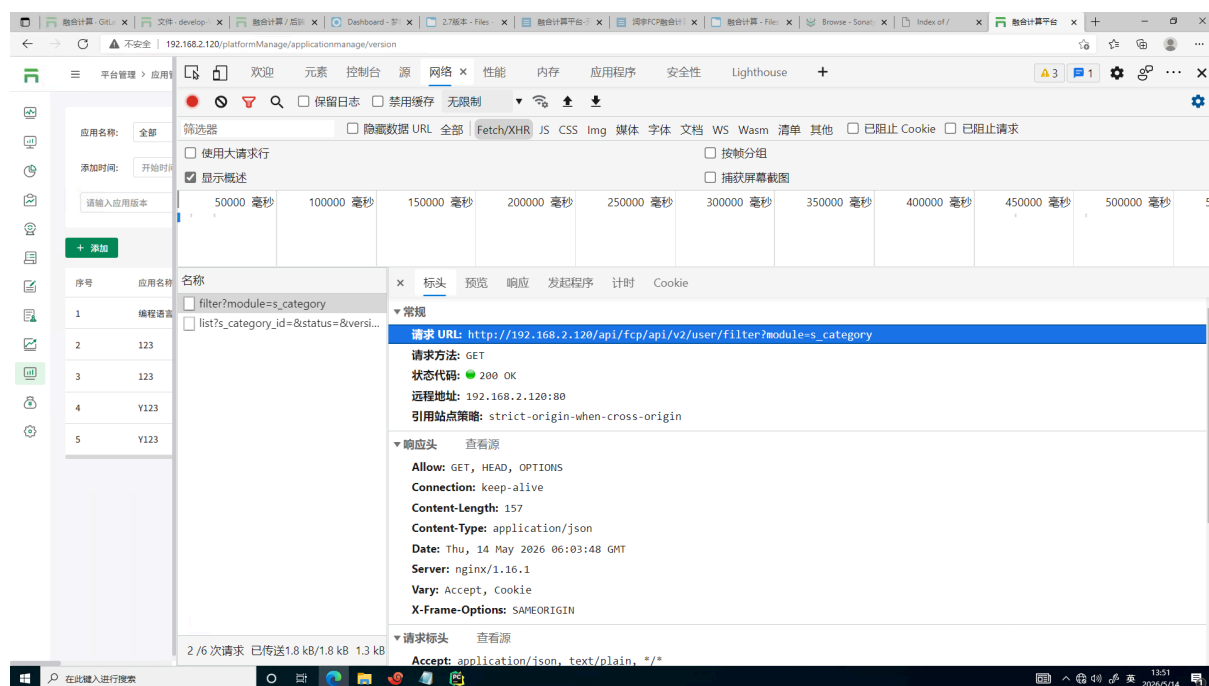
- **设计意图：**防止管理员误将一个公开的应用移动到一个隐藏的分类下，导致前端数据展示逻辑冲突。

4. 总结

这个“移动”操作实际上完成了一次深拷贝式的逻辑迁移：

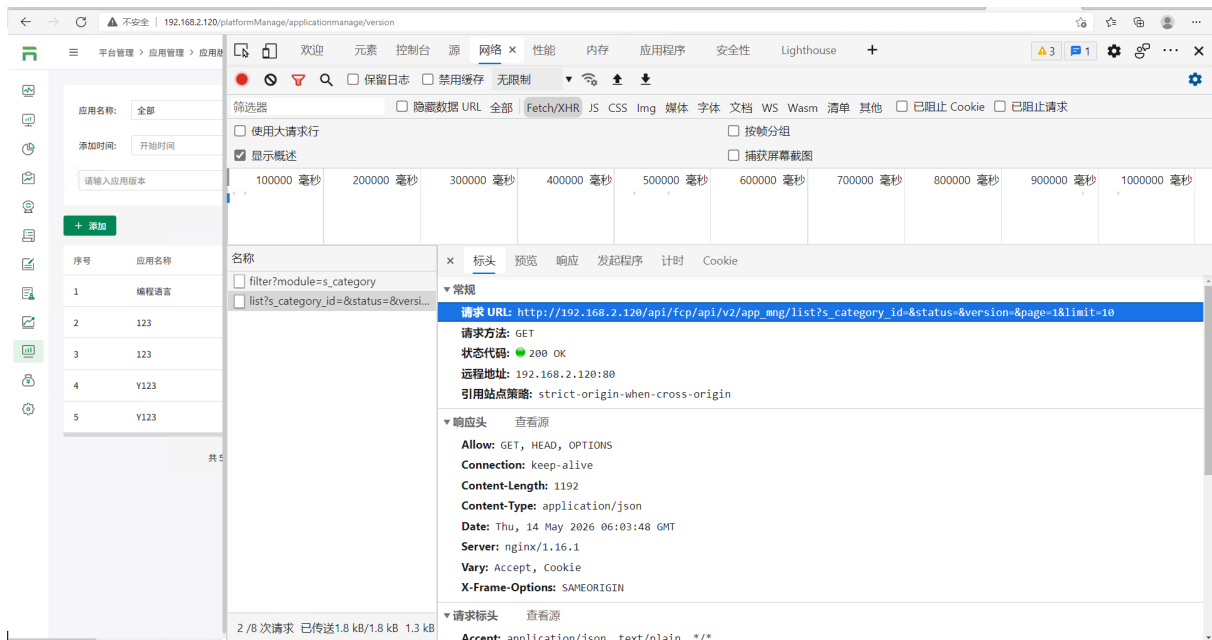
1. 它改变了应用的归属关系（`pid`）。
2. 它根据目的地的“环境”（大类状态），自动修正了应用及其版本的所有权限。
3. 它通过事务确保了操作的原子性。

第二类：应用版本管理



它通过这一个入口统一管理了全平台几十个页面的所有下拉框逻辑。你会发现代码里还有 `bill`（账单）、`ai_job`（AI 任务）、`device`（设备）等分支，都是通过这一个接口支持的。（和自定义源码不相关）

一 获取应用版本列表接口（分页，多维度筛选）list GET



views.py

```
class ListView(APIView):
    @method_decorator(permission(ustertype=["admin"], permission="plat_set_get"))
    def get(self, request):
        r = AppList().get(request)
        return Response(r)
```

1. 权限与入口：该接口是**管理员专用**的

2. 核心逻辑逻辑：主要在 `AppList.py` 的 `get` 方法中，采用了典型的“查表 → 内存组装 → 返回”模式。

A. 参数解析与校验

- 它从 URL 中提取 `s_category_id`（分类）、`status`（状态）、`version`（版本号）以及分页参数 `page` 和 `limit`。

```
class AppList(object):
```



```

def get(self, request):
    s_category_id = request.GET.get("s_category_id", None)
    status = request.GET.get("status", None)
    version = request.GET.get("version", None)
    s_time = request.GET.get("s_time", None)
    e_time = request.GET.get("e_time", None)

    result = {
        "code": -1,
        "message": "failed"
    }
    if status not in ["", None, "0", 0, "1", 1]:
        result["err"] = "未知的状态"
        return result

    page_index = request.GET.get("page", 1)
    page_size = request.GET.get("limit", 10)

    if page_index in ["", None]:
        page_index = 1
    if page_size in ["", None]:
        page_size = 10

    page_index = int(page_index)
    page_size = int(page_size)

    data = {
        "page": page_index,
        "limit": page_size,
        "count": 0,
        "app_version": []
    }

```

- **细节：**它会对 `status` 进行合法性检查，如果不是预设的几个值（0, 1 等），直接返回“未知状态”。

B. 多表关联与筛选

三张表的逻辑：

1. **TbAppCategory (分类表)**: 先查出所有的分类信息, 存进内存字典 (`ac_dict`)。

```
ac_obj = TbAppCategory.objects.filter(is_extern=1).exclude(pid=0)
if s_category_id not in ['', None]:
    ac_obj = ac_obj.filter(app_category_id=s_category_id)
```

分别获取全部的和主类信息

```
# ac_val = ac_obj.values("app_category_id", "category_name")
ac_val = ac_obj.values("app_category_id", "category_name", "status", "pid")
ac_dict = {}
ac_d_dict = {}
pid_list = []
for ac in ac_val:
    category_name = ac["category_name"]
    category_id = ac["app_category_id"]

    ac_dict[category_id] = category_name

    ac_d_dict[category_id] = {
        "pid": ac["pid"],
        "status": ac["status"]
    }
    pid_list.append(ac["pid"])

# 主类信息
pid_dict = {}
ac_p_obj = TbAppCategory.objects.filter(is_extern=1, pid=0)
ac_p_val = ac_p_obj.values("app_category_id", "category_name", "status")
for ac_p in ac_p_val:
    p_category_name = ac_p["category_name"]
    p_category_id = ac_p["app_category_id"]

    pid_dict[p_category_id] = {
        "category_name": p_category_name,
        "status": ac_p["status"]
    }

category_id_list = ac_dict.keys()
```

1. **TbAppVersion (版本表)**: 这是主表。后端根据你传入的分类、状态、版本号, 拼接成 SQL 的 `filter` 条件。
 - **时间筛选**: 代码里用 `extra(where=[cond])` 手写了一段原生 SQL 来处理 `create_time` 的范围查询 (`BETWEEN ... AND ...`)。

```

app_obj = TbAppVersion.objects.filter(is_extern=1, category_id__in=category_id_list)
if status not in ["", None]:
    app_obj = app_obj.filter(status=status)
if version not in ["", None]:
    app_obj = app_obj.filter(version=version)

if s_time not in ["", None] and e_time not in ["", None]:
    cond = f"create_time BETWEEN '{s_time}' AND '{e_time}'"
    app_obj = app_obj.extra(where=[cond])

val = app_obj.values("category_id", "app_version_id", "version", "app_path",
                    "create_time", "status").order_by("-create_time")

```

1. **TbUserApp (关联表)**：在循环每一行数据时，它会实时查一次这张表：

`TbUserApp.objects.filter(app_version_id=...).count()` **关键逻辑**：这行代码计算了“授权用户数”。也就是说，表格里那一列动态数字，就是在这里实时算出来的。

```

        total_num = len(val)

        page_num = int(total_num / page_size)
        else_num = total_num % page_size
        if else_num > 0:
            page_num = page_num + 1

        if page_index > page_num:
            page_index = page_num

        d_i_s = page_size * (page_index - 1)
        d_i_e = page_size * page_index

        if d_i_s < 0:
            d_i_s = 0

        val = val[d_i_s:d_i_e]

    for i in val:
        category_id = i["category_id"]

        app_version_id = i["app_version_id"]
        version = i["version"]
        app_path = i["app_path"]
        create_time = i["create_time"]
        status = i["status"]

        bind_user_num = TbUserApp.objects.filter(app_version_id=app_version_id)\
            .values_list("user_id", flat=True).distinct().count()

        cty_name = None
        cty_status = None

```

```

        pid = ac_d_dict[category_id]["pid"]
        if pid in pid_dict.keys():
            cty_name = pid_dict[pid]["category_name"]
            cty_status = pid_dict[pid]["status"]

        ad = {
            "s_category_name": ac_d_dict[category_id],
            "app_version_id": app_version_id,
            "version": version,
            "bind_user_num": bind_user_num,
            "position": app_path,
            "create_time": str(create_time),
            "status": status,
            "category_name": cty_name,
            "category_status": cty_status,
            "s_category_status": ac_d_dict[category_id]["status"]
        }

        data["app_version"].append(ad)

    data["count"] = total_num

    result = {
        "code": 0,
        "message": "success",
        "data": data
    }

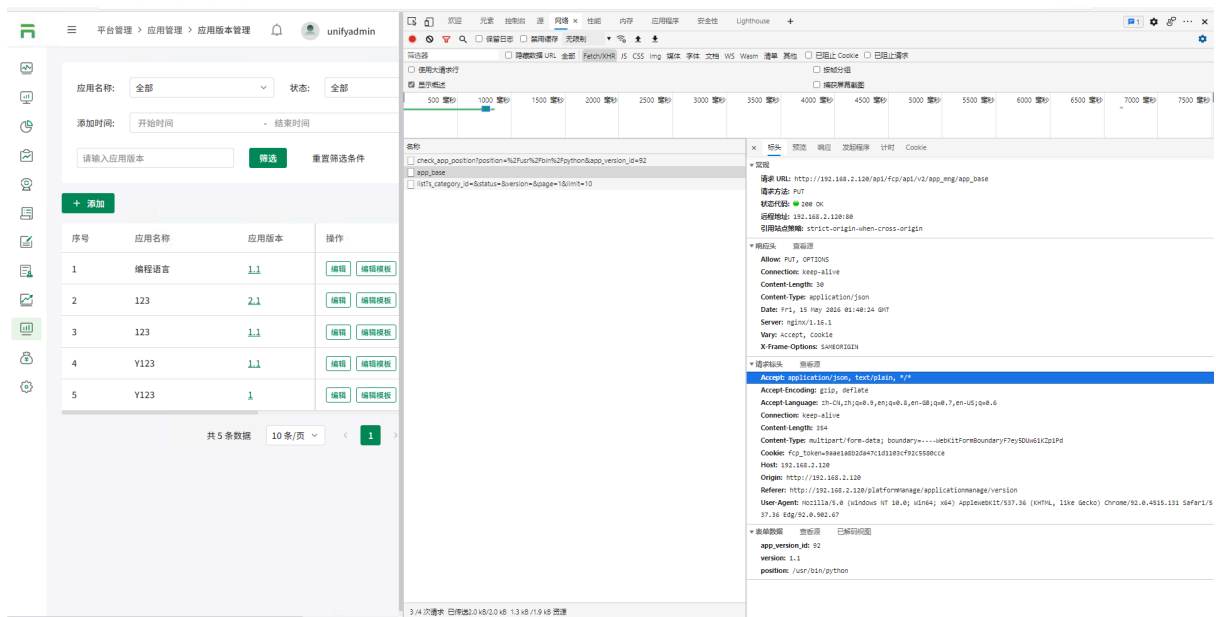
    return result

```

C. 分页与数据封装

- 它并没有使用 Django 自带的 Paginator，而是通过计算 `page_index` 和 `page_size` 对查询结果集进行切片（`val[d_i_s:d_i_e]`）。
- 最后把分类名称、授权用户数、文件路径（`app_path`）等拼成一个大字典返回。

二 编辑应用版本基本信息app_base PUT



views.py

```
class AppBaseView(APIView):
    @method_decorator(permission(user_type=["admin"], permission="plat_set_op"))
    def put(self, request):
        r, event = AppBase().put(request)
        # 记录日志
        if r["code"] == 0 and event is not None:
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                       operate_type="编辑应用信息",
                                       operate_content=event, state=True)
        return Response(r)
```

1.

```

10 class AppBase(object):
11     def put(self, request):
12         app_version_id = request.POST.get("app_version_id", None)
13         version = request.POST.get("version", None)
14         app_path = request.POST.get("position", None)
15
16         event = None
17         result = {
18             "code": -1,
19             "message": "failed"
20         }
21
22         if app_version_id in ["", None]:
23             result["err"] = "请指定应用版本id"
24             return result, event
25         if version in ["", None]:
26             result["err"] = "请指定应用版本"
27             return result, event
28         if app_path in ["", None]:
29             result["err"] = "请指定应用位置"
30             return result, event
31
32         obj = TbAppVersion.objects.filter(is_extern=1, app_version_id=app_version_id)
33         if obj.count() == 0:
34             result["err"] = "应用版本不存在"
35             return result, event
36
37         val = obj.values()
38         bf_version = val[0]["version"]
39         bf_app_path = val[0]["app_path"]

```

获取并检查数据

```

        cg_flag_dict = {
            "version": False,
            "app_path": False
        }
        if bf_version != version:
            cg_flag_dict["version"] = True
        if bf_app_path != app_path:
            cg_flag_dict["app_path"] = True

        # 无修改
        if True not in cg_flag_dict.values():
            result = {
                "code": 0,
                "message": "success"
            }
            return result, event

        category_id = val[0]["category_id"]

        c_obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)
        if c_obj.count() == 0:
            result["err"] = "应用不存在"
            return result, event

        category_name = c_obj.values("category_name")[0]["category_name"]
        lic_name = f"{category_name}_{version}"

        ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

```

保留修改的项那一套

1. 变更检测机制 (`cg_flag_dict`)

代码延续了该项目优良的性能优化习惯：

- **静默处理**：后端会对比数据库里的旧数据 (`bf_version` , `bf_app_path`) 和前端传来的新数据。
- **零改动跳过**：如果没有发生任何变化，直接返回成功，不触发数据库写操作和事务，节省服务器资源。

2. 核心功能：双表联动更新

这是该代码中最关键的部分。


```

    with transaction.atomic():
        ver_u_data = {
            "version": version,
            "app_path": app_path,
            "update_time": ctime
        }
        obj.update(**ver_u_data)

        lic_u_data = {
            "lic_name": lic_name,
            "update_time": ctime
        }
        l_obj = TbAppLicense.objects.filter(is_bind=1, app_version_id=app_version_id)
        l_obj.exclude(lic_name=lic_name).update(**lic_u_data)

```

- **事务保护**：代码使用了 `transaction.atomic()`。这意味着对“版本表”和“授权表”的修改必须同时成功，否则全部回滚。
- **不仅仅改路径**：当你修改了 `version`（版本号）时，系统会自动重新生成 `lic_name`（授权名称，格式为：`类别名_版本号`），并同步更新到 `AppLicense` 表中。
- **目的**：确保授权信息与版本信息始终保持一致。如果只改了版本号而没改授权名，后续的授权校验就会失效。

3. 高度细化的审计日志

```

# 日志
from common.config.project import LOG_PREFIX

#
# op_obj = "应用名称: %s" % category_name
# log_bef = "%s应用版本: %s; 应用位置: %s" % (LOG_PREFIX["bef"], val["version"], val["app_path"])
# log_aft = "%s应用版本: %s; 应用位置: %s" % (LOG_PREFIX["aft"], version, app_path)
#
# event = "%s\n%s\n%s" % (op_obj, log_bef, log_aft)

op_obj = '''<span class="log_log_obj"><span class="obj">应用名称: </span><span class="val"></span></span></p>''' % category_name
log_bef = '''<span class="log_log_content_bef"><span class="bef"></span><span class="obj">应用版本: </span><span class="val"></span><span class="obj">应用位置: </span><span class="val"></span></span></p>''' \
    % (LOG_PREFIX["bef"], bf_version, bf_app_path)

version_css = ""
app_path_css = ""
if cg_flag_dict["version"]:
    version_css = "changed_val"
if cg_flag_dict["app_path"]:
    app_path_css = "changed_val"
log_aft = '''<span class="log_log_content_aft"><span class="aft"></span><span class="obj">应用版本: </span><span class="val"></span><span class="obj">应用位置: </span><span class="val"></span></span></p>''' \
    % (LOG_PREFIX["aft"], version_css, version, app_path_css, app_path)

event = "%s\n%s\n%s" % (op_obj, log_bef, log_aft)

result = {
    "code": 0,
    "message": "success"
}

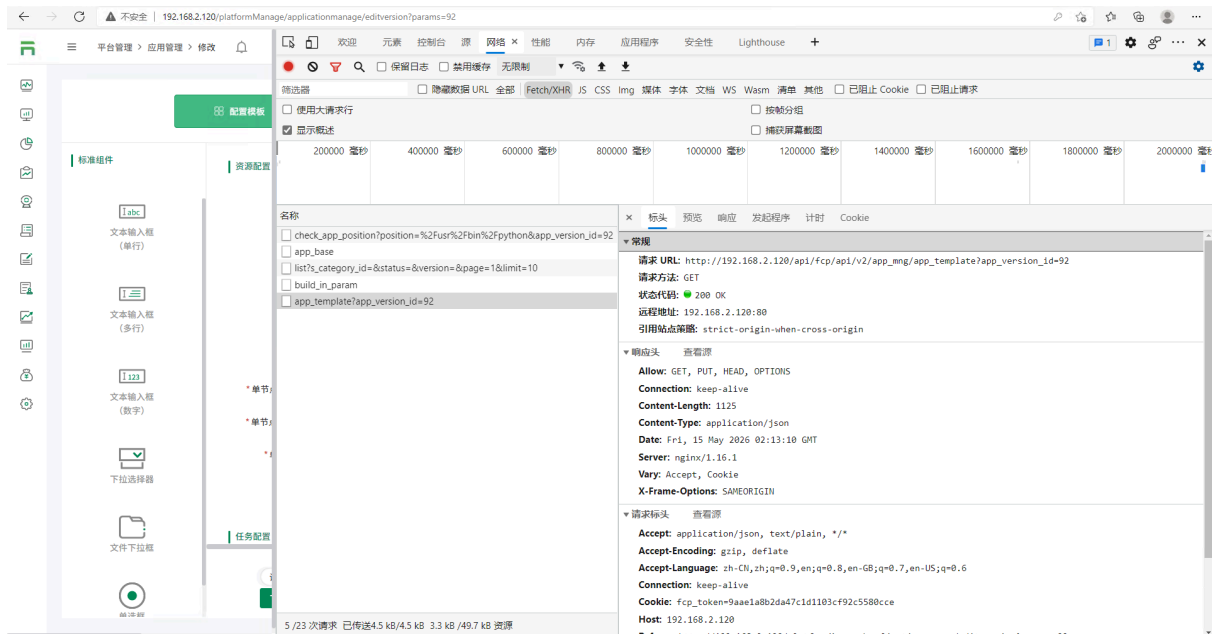
return result, event

```

日志生成的逻辑非常繁琐但很有用（见 `image_315ca5.png`）：

- **可视化差异**：它不仅记录“谁改了应用”，还通过构造 HTML 片段记录了修改前后的具体值。
- **高亮显示**：利用 `changed_val` 这个 CSS 类，管理员在后台日志中能一眼看到哪个字段变红了（变动了）。

三 修改模板和读取模板app_template



views.py

```
class AppTemplateView(APIView):
    @method_decorator(permission(user_type=["admin"], permission="plat_set_op"))
    def put(self, request):
        r, event = AppTemplate().put(request)
        # 记录日志
        if r["code"] == 0:
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                       operate_type="编辑应用模板",
                                       operate_content=event, state=True)
        return Response(r)

    @method_decorator(permission(user_type=["admin"], permission="plat_set_get"))
    def get(self, request):
        r = AppTemplate().get(request)
        return Response(r)
```

读取模板 GET

- 权限：仅限 `admin` 用户，且需要 `plat_set_get`（查询）权限。

这是当你点击“编辑模板”按钮，弹窗加载出已有数据时执行的操作。

```

83     def get(self, request):
84         app_version_id = request.GET.get("app_version_id", None)
85
86         result = {
87             "code": -1,
88             "message": "failed"
89         }
90         if app_version_id in ["", None]:
91             result["err"] = "请指定应用版本id"
92             return result
93
94         obj = TbAppTemplate.objects.filter(app_version_id=app_version_id)
95         if obj.count() == 0:
96             result["err"] = "应用模板不存在"
97             return result
98
99         val = obj.values("app_template_id", "param_data", "script_exec", "is_remote", "script_remote", "create_time")
100         at_data = {
101             "app_template_id": val[0]["app_template_id"],
102             "param_data": val[0]["param_data"],
103             "script_exec": val[0]["script_exec"],
104             "is_remote": val[0]["is_remote"],
105             "script_remote": val[0]["script_remote"],
106             "create_time": str(val[0]["create_time"])
107         }
108
109         result = {
110             "code": 0,
111             "message": "success",
112             "data": {"app_template": at_data}
113         }
114
115         return result

```

- 核心逻辑类： `AppTemplate().get(request)`
- 关键过程：
 1. 定位：根据前端传来的 `app_version_id`（应用版本ID）去数据库查询。
 2. 校验：如果该版本没有关联的模板，会报错“应用模板不存在”。
 3. 数据组装：如果存在，它会把 `param_data`（参数数据）、`script_exec`（执行脚本）、`is_remote`（是否远程）、`script_remote`（远程脚本）等字段全部打包返回给前端。

修改/保存模板 PUT

- 权限：仅限 `admin` 用户，且需要 `plat_set_op`（操作）权限。

这是你在弹窗里改完配置，点击“保存”按钮时执行的操作。

- 核心逻辑类： `AppTemplate().put(request)`
- 关键过程：

```

9 class AppTemplate(object):
10     def put(self, request):
11         app_version_id = request.POST.get("app_version_id", None)
12         param_data = request.POST.get("param_data", None)
13         is_remote = request.POST.get("is_remote", None)
14         script_exec = request.POST.get("script_exec", None)
15         script_remote = request.POST.get("script_remote", None)
16
17         event = None
18         result = {
19             "code": -1,
20             "message": "failed"
21         }
22
23         if app_version_id in ["", None]:
24             result["err"] = "请指定应用版本id"
25             return result, event
26         if param_data in ["", None]:
27             result["err"] = "请指定应用参数"
28             return result, event
29         if is_remote not in [0, "0", 1, "1"]:
30             result["err"] = "请指定交互应用状态"
31             return result, event
32         if script_exec in ["", None]:
33             result["err"] = "请指定Script脚本"
34             return result, event
35
36         obj = TbAppTemplate.objects.filter(app_version_id=app_version_id)
37         if obj.count() == 0:
38             result["err"] = "应用模板不存在"
39             return result, event
40
41         ver_obj = TbAppVersion.objects.filter(is_extern=1, app_version_id=app_version_id)
42         if ver_obj.count() == 0:
43             result["err"] = "应用版本不存在"
44             return result, event
45
46         ver_val = ver_obj.values("category_id", "version")
47         category_id = ver_val[0]["category_id"]
48         version = ver_val[0]["version"]

```

```

49
50         cty_obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)
51         if cty_obj.count() == 0:
52             result["err"] = "应用不存在"
53             return result, event
54
55         category_name = cty_obj.values("category_name")[0]["category_name"]

```

1. **严格校验：**代码会检查参数数据、脚本内容等是否为空，并确认对应的应用版本和分类确实存在。

```

55     category_name = cty_obj.values("category_name")[0]["category_name"]
56
57     ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
58     script_exec = script_exec.replace("\r\n", "\n")
59     data = {
60         "param_data": param_data,
61         "script_exec": script_exec,
62         "is_remote": is_remote,
63         "update_time": ctime
64     }
65
66     if script_remote is not None:
67         script_remote = script_remote.replace("\r\n", "\n")
68         data["script_remote"] = script_remote
69
70     obj.update(**data)

```

1. **文本清洗 (小细节)**: 代码中有一行 `script_exec.replace("\r\n", "\n")`。这是为了统一换行符（将 Windows 的 `\r\n` 转为 Unix 的 `\n`），防止脚本在服务器上执行时因为换行符问题报错。
2. **持久化**: 使用 `obj.update(data)` 更新数据库中的模板记录，并刷新 `update_time`。

```

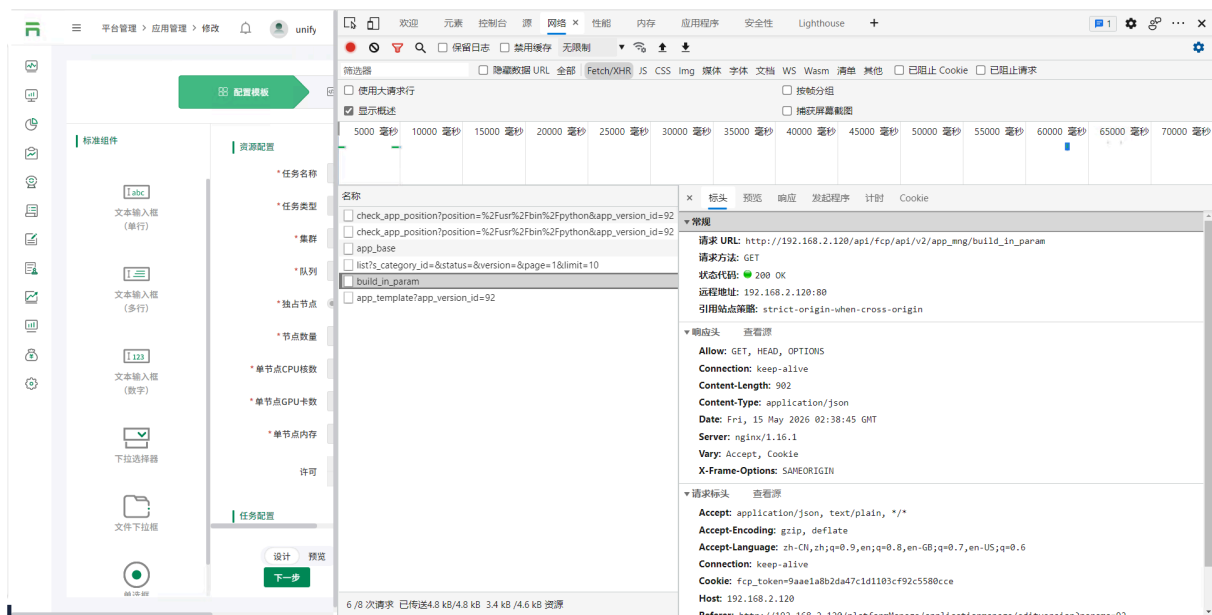
# event = f"应用名称: {category_name}; 应用版本: {version}"
event = '''<p class="log_log_obj"><span class="obj">应用名称:
</span><span class="val">%s</span>; <span class="obj">应用版本: </span><span class="val">%s</span></p>''' \
    % (category_name, version)
result = {
    "code": 0,
    "message": "success"
}

return result, event

```

1. **详细日志**: 保存成功后，系统会记录一条审计日志，格式为 `应用名称: xxx; 应用版本: xxx`，方便后续追查是谁改了配置。

四 将后端预定义的配置（静态数据）打包喂给前端（关系不大）



views.py

```
class BuildInParameterView(APIView):
    @method_decorator(permission(usertype=["admin"], permission="plat_set_get"))
    def get(self, request):
        job_type = request.GET.get('job_type')
        r = build_in_param()
        return Response(r)
```

- **权限控制**：只有管理员（`admin`）在进行平台设置（`plat_set_get`）时才能拉取到这些配置。

1. 干什么用的？

在上一节我们看的“应用模板”里，用户可能需要填写一些脚本或参数。为了方便用户，系统通常会提供一些**内置变量**（比如：`$USER_ID`、`$APP_PATH`、`$PORT` 等）。

- 这个接口就是把这些后端能识别的“内置变量列表”告诉前端。
- 前端拿到这些数据后，通常会把它们显示在下拉框里，或者作为输入框的提示信息。

2. 代码逻辑拆解

- **数据来源**：注意第一行导包：`from common.config.app import VARS_BUILD_IN`。这说明数据根本没存数据库，而是直接写死在后端的配置文件里的。

```

VARS_BUILD_IN = {
    "${APP_PATH}": {"description": "应用位置"},
    "${job_name}": {"description": "作业名称"},
    "${work_path}": {"description": "工作目录"},
    "${queue}": {"description": "运行队列"},
    "${runtime_limit}": {"description": "运算时限"},
    "${node_num}": {"description": "节点数量"},
    "${ncpu_per_node}": {"description": "每节点CPU"},
    "${ngpu_per_node}": {"description": "每节点GPU"},
    "${mem_mb_per_node}": {"description": "每节点内存(MB)"},
    "${exclusive}": {"description": "独占节点"},
    "${exec_filename}": {"description": "启动文件"},
    "${additional_param}": {"description": "应用附加参数"},
    "${lic_num}": {"description": "许可数量"}, ##
    "${network}": {"description": "网络 (例如: ib) "}, ##
    "${ntasks_per_node}": {"description": "每节点任务数"},
    "${cpus_per_task}": {"description": "每任务CPU数"},
    "${cpu_per_node_type}": {"description": "每节点CPU使用类型"}
}

MARK_DATA = [
    "使用ncpu_per_node时, 不可使用ntasks_per_node和cpus_per_task;
    使用ntasks_per_node和cpus_per_task时, 不可使用ncpu_per_node。
    ntasks_per_node和cpus_per_task是一组参数, 只会同时存在或同时不存在"
]

```

- **零业务逻辑**：函数内部没有任何增删改查，只是创建了一个标准的 JSON 结构，把 `VARS_BUILD_IN` 塞进 `data` 字段里就返回了。

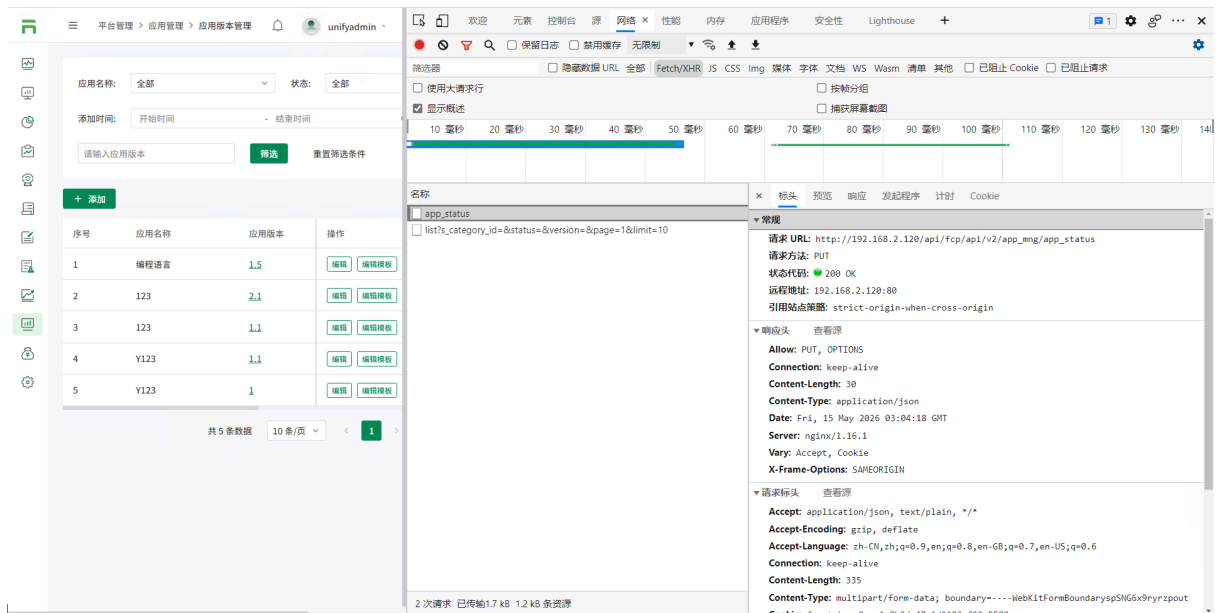
```
build_in_param.py
FCP_Project\urls.py x app_mng\urls.py x views.py x build_in_param.py x AppTemp
1  #-*- coding: utf-8 -*-
2  # @Author: ouyang
3  # @Time: 2024/5/22 14:51
4  from common.config.app import VARS_BUILD_IN
5
6  def build_in_param():
7      output = {
8          "code": 0,
9          "message": "success",
10         "data": VARS_BUILD_IN
11     }
12
13     return output
14
```

3. 架构视角：为什么不让前端自己写死？

你可能会问：既然是写死的配置，为什么前端不自己维护一份？

- **单源真理 (Single Source of Truth)**：如果后端脚本逻辑更新了，增加了新的内置变量，开发者只需要改后端的配置文件，前端就会自动更新，不需要重新发版。
- **前后端解耦**：前端不需要关心这些变量具体代表什么，它只负责展示后端给出的“合法选项”。

五 发布下架应用app_status PUT



views.py

```
class AppStatusView(APIView):
    @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
    def put(self, request):
        r, event = AppStatus().put(request)
        # 记录日志
        if r["code"] == 0 and event is not None:
            op_type = event.split(" -- ")[0]
            op_type = f"{op_type}应用版本"
            event = event.split(" -- ")[1]
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                       operate_type=op_type,
                                       operate_content=event, state=True)
        return Response(r)
```

1. 核心功能：应用版本的状态切换

该接口通过 **PUT** 请求，控制某个特定版本（比如“编程语言”应用的 1.5 版本）是否对用户可见。

2. 关键输入参数

```

58     def put(self, request):
59         app_version_id = request.POST.get("app_version_id", None)
60         status = request.POST.get("status", None)
61         verify = request.POST.get("verify", None)
62
63         event = None
64         result = {
65             "code": -1,
66             "message": "failed"
67         }
68
69         if app_version_id in ["", None]:
70             result["err"] = "请指定应用版本id"
71             return result, event
72         if status in ["", None] and status not in [0, 1, "0", "1"]:
73             result["err"] = "请确定发布/下架"
74             return result, event
75
76         obj = TbAppVersion.objects.filter(is_extern=1, app_version_id=app_version_id)
77
78         if obj.count() == 0:
79             result["err"] = "应用版本不存在"
80             return result, event
81
82         val = obj.values("category_id", "version")
83         category_id = val[0]["category_id"]
84         version = val[0]["version"]
85         ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

```

- **app_version_id**：指定要操作的那个版本。
- **status**：**1** 表示发布，**0** 表示下架。
- **verify**（重要新参数）：
 - **0**：检测模式。后端会先检查路径、分类状态等是否允许上架。
 - **1**：强制模式。直接执行上架逻辑，并强制修正关联状态。

3. 后端处理的核心逻辑（级联发布）

这是该接口最有技术含量的部分：

A. 发布逻辑（**status=1**）—— 向上“开门”

当你发布一个应用版本时，系统会为了保证用户能看到它，自动执行一系列向上追溯的动作(上架状态全部打开)：

```

    if int(status) == 1: # 发布
        if verify in ["", None] and verify not in [0, 1, "0", "1"]:
            result["err"] = "请确定检测/强制上架"
            return result, event

        if obj.filter(status=0).count() == 0:
            result["err"] = "应用版本已发布"
            return result, event

        if int(verify) == 0: # 检测
            result = self.get_category_status(category_id)

        else: # 强制上架
            op_type = "发布"

            from django.db import transaction

            with transaction.atomic():
                obj.update(status=1, update_time=ctime)

                cty_obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)
                if cty_obj.count() == 0:
                    result["err"] = "应用不存在"
                    return result, event

                cty_val = cty_obj.values("category_name", "pid", "status")
                category_name = cty_val[0]["category_name"]
                pid = cty_val[0]["pid"]
                c_status = cty_val[0]["status"]

```

```

        if int(c_status) == 0:
            cty_obj.update(status=1, update_time=ctime)

            p_cty_obj = TbAppCategory.objects.filter(is_extern=1, pid=0, app_category_id=pid)
            if p_cty_obj.count() == 0:
                result["err"] = "分类不存在"
                return result, event

            p_c_status = p_cty_obj.values("status")[0]["status"]

            if int(p_c_status) == 0:
                p_cty_obj.update(status=1, update_time=ctime)

            # event = f"{op_type} -- 应用名称: {category_name}; 应用版本: {version}"
            event = '''<p class="log log_obj"><span class="obj">应用名称:
</span><span class="val">%s</span>; <span class="obj">应用版本: </span><span class="val">%s</span></p>''' \
                % (category_name, version)
            event = "%s -- %s" % (op_type, event)

            result = {
                "code": 0,
                "message": "success"
            }

```

1. **更新版本表**：先将 `TbAppVersion` 的状态改为 1。
2. **检查并开启“应用”**：找到该版本所属的应用（小类）。如果这个应用目前是下架的，系统会自动把它**改为发布状态**。

3. **检查并开启“大类”**：继续向上找，找到该应用所属的大类。如果大类也是下架的，系统会强制把大类也发布了。

逻辑总结：只要你敢发布一个最小的版本，系统就敢帮你把通往这个版本的“大门”（应用）和“院门”（分类）全部打开。

代码使用了 `transaction.atomic()`。这意味着修改必须同时成功，否则全部回滚。

B. 下架逻辑（`status=0`）—— 精确关停

```
140         else: # 下架
141             if obj.filter(status=1).count() == 0:
142                 result["err"] = "应用版本已下架"
143                 return result, event
144
145             val = obj.values("category_id", "version")
146             category_id = val[0]["category_id"]
147             version = val[0]["version"]
148             cty_obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)
149
150             if cty_obj.count() == 0:
151                 result["err"] = "应用不存在"
152                 return result, event
153
154             op_type = "下架"
155
156             category_name = cty_obj.values("category_name")[0]["category_name"]
157
158             obj.update(status=0, update_time=ctime)
159
160             # event = f"{op_type} -- 应用名称: {category_name}; 应用版本: {version}"
161             event = '''<p class="log log_obj"><span class="obj">应用名称:
162             </span><span class="val">%s</span>; <span class="obj">应用版本: </span><span class="val">%s</span></p>''' \
163                 % (category_name, version)
164             event = "%s -- %s" % (op_type, event)
165             result = {
166                 "code": 0,
167                 "message": "success"
168             }
169             return result, event
```

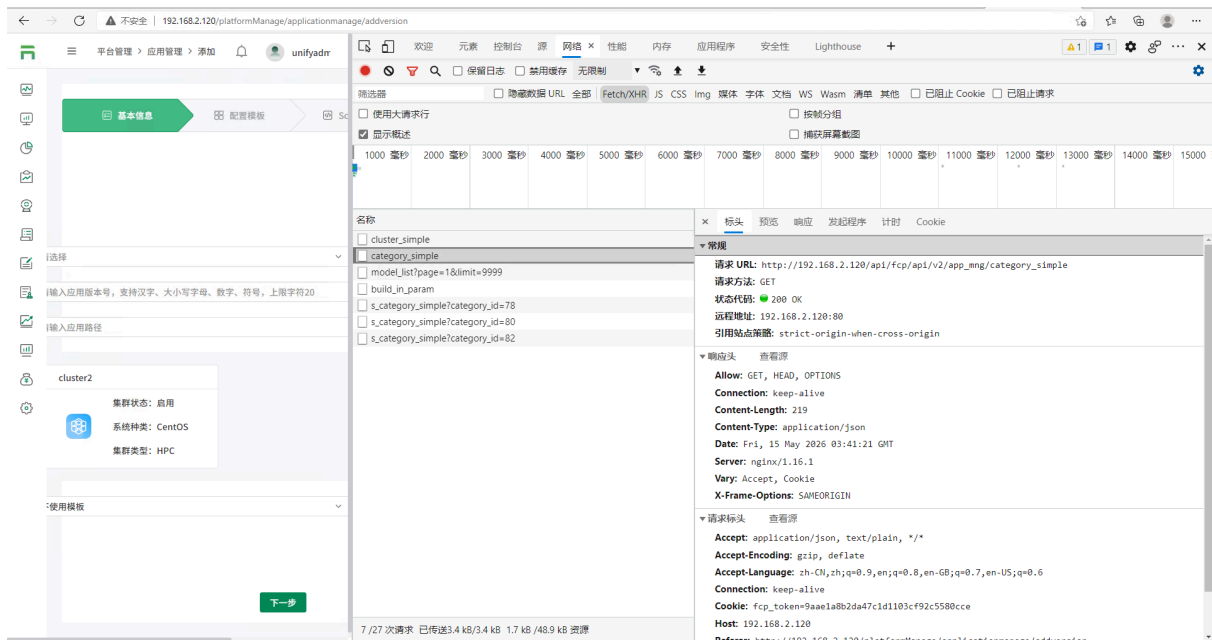
下架逻辑相对简单：

- **操作**：仅将该版本的状态设为 0。
- **注意**：下架一个版本不会自动下架应用或分类。因为一个应用可能有多个版本，你关掉 1.5 版本，用户可能还在用 1.1 版本。

分析代码以及经过实验可得：

应用版本上架有全部上架权限，应用分类下架大类有全部权限，小类有单个应用下架权限。

六大类字典加载category_simple GET



views.py

```
class CategorySimpleView(APIView):
    @method_decorator(permission(user_type=["admin"], permission="plat_set_get"))
    def get(self, request):
        r = Category().get_simple(request)
        return Response(r)
```

1. 核心功能：大类字典加载

当管理员打开“添加应用版本”页面时，第一个下拉框（应用名称）需要知道系统里有哪些大类可选。这个接口就是用来拉取这些“入场券”的。

2. 关键逻辑拆解

- 身份验证：
 - 依然严格限制为 `admin` 用户。
 - 需要 `plat_set_get`（平台设置查询）权限。

```

9
10 class Category(object):
11     def get_simple(self, request):
12         category = []
13
14         obj = TbAppCategory.objects.filter(is_extern=1, pid=0)
15         val = obj.values("app_category_id", "category_name", "status")
16         if len(val) > 0:
17             for i in val:
18                 d = {
19                     "app_category_id": i["app_category_id"],
20                     "category_name": i["category_name"],
21                     "status": i['status']
22                 }
23                 category.append(d)
24
25         data = {"category": category}
26         result = {
27             "code": 0,
28             "message": "success",
29             "data": data
30         }
31         return result

```

- **筛选条件（重点）：**

- `is_extern=1`：只查询外部应用类别。
- `pid=0`：这是该接口的“灵魂”。它确保只获取**顶级大类**。在数据库设计中，`pid=0` 代表它没有父节点，即处于树状结构的最高层。

- **精简数据：**

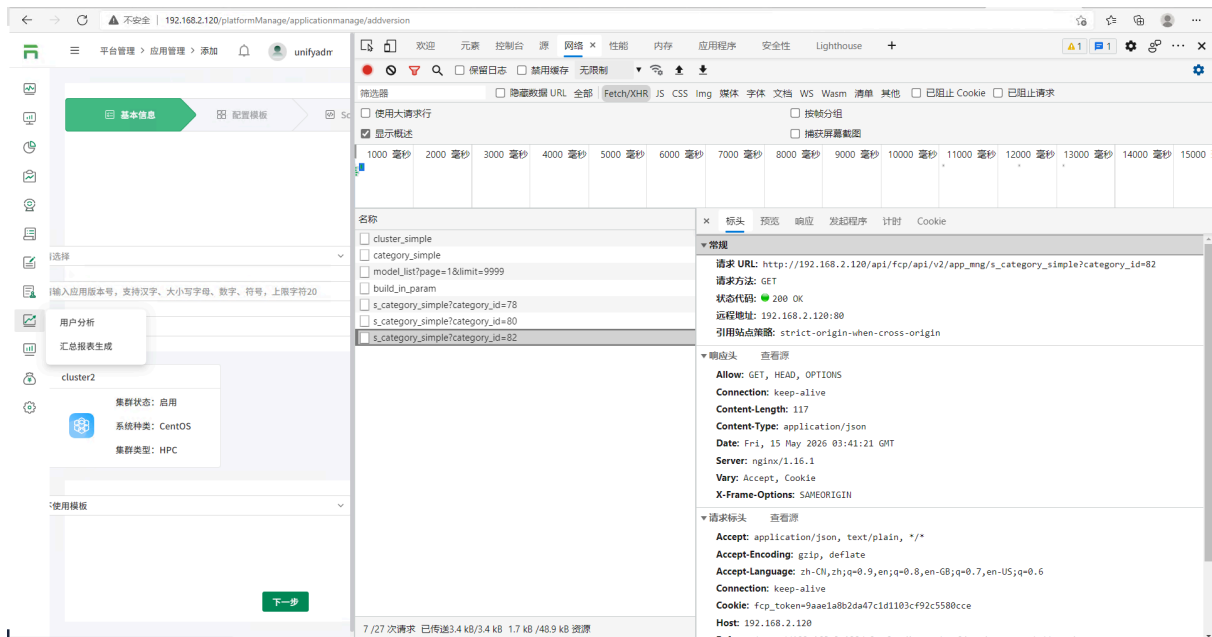
- 它只取了三个字段：`app_category_id`（ID）、`category_name`（名称）、`status`（状态）。
- 这种“Simple”设计是为了减少网络开销，因为下拉框不需要简介、图标或创建时间这些额外信息。

3. 业务意义

这个接口体现了“分级加载”的策略：

1. **第一步**：调用这个接口，让用户选一个“大类”（比如：编程语言）。
2. **第二步**：当用户选好大类后，前端会拿到这个大类的 ID，再去调用下一个接口（通常是 `s_category_simple`）来获取该大类下的具体“小应用”。

七 小类字典加载s_category_simple GET



views.py

```
class SCategorySimpleView(APIView):
    @method_decorator(permission(user_type=["admin"], permission="plat_set_get"))
    def get(self, request):
        r = CategoryApp().get_simple(request)
        return Response(r)
```

1. 核心功能：子类（小类）字典加载

当你通过上一个接口选好了“大类”（Parent Category）后，前端会立即调用这个接口。它的作用是：根据你选的大类 ID，找出该类目下所有可以添加版本的具体“小应用”。

2. 关键逻辑拆解

```

def get_simple(self, request):
    pid = request.GET.get("category_id", None)

    result = {
        "code": -1,
        "message": "failed"
    }

    if pid in ["", None]:
        result["err"] = "请指定类别id"
        return result

    category = []

```

- **级联绑定 (pid):**

- 代码通过 `pid = request.GET.get("category_id")` 获取父类 ID。
- **强制校验:** 如果请求里没带 `category_id`，接口会直接报错“请指定类别id”。这确保了数据的逻辑严密性——没有大类，就不可能有小类。

```

obj = TbAppCategory.objects.filter(is_extern=1, pid=pid)
val = obj.values("app_category_id", "category_name", "status")
if len(val) > 0:
    for i in val:
        d = {
            "s_category_id": i["app_category_id"],
            "category_name": i["category_name"],
            "status": i["status"]
        }
        category.append(d)

data = {"s_category": category}
result = {
    "code": 0,
    "message": "success",
    "data": data
}
return result

```

- **精准过滤:**

- `TbAppCategory.objects.filter(is_extern=1, pid=pid)`：这里是核心。它去数据库里寻找所有“父 ID 等于刚才选的大类 ID”的记录。

- **轻量化返回:**

- 和主分类一样，它只返回 ID、名称和状态。这种设计非常适合前端的 `Select` 组件直接渲染。

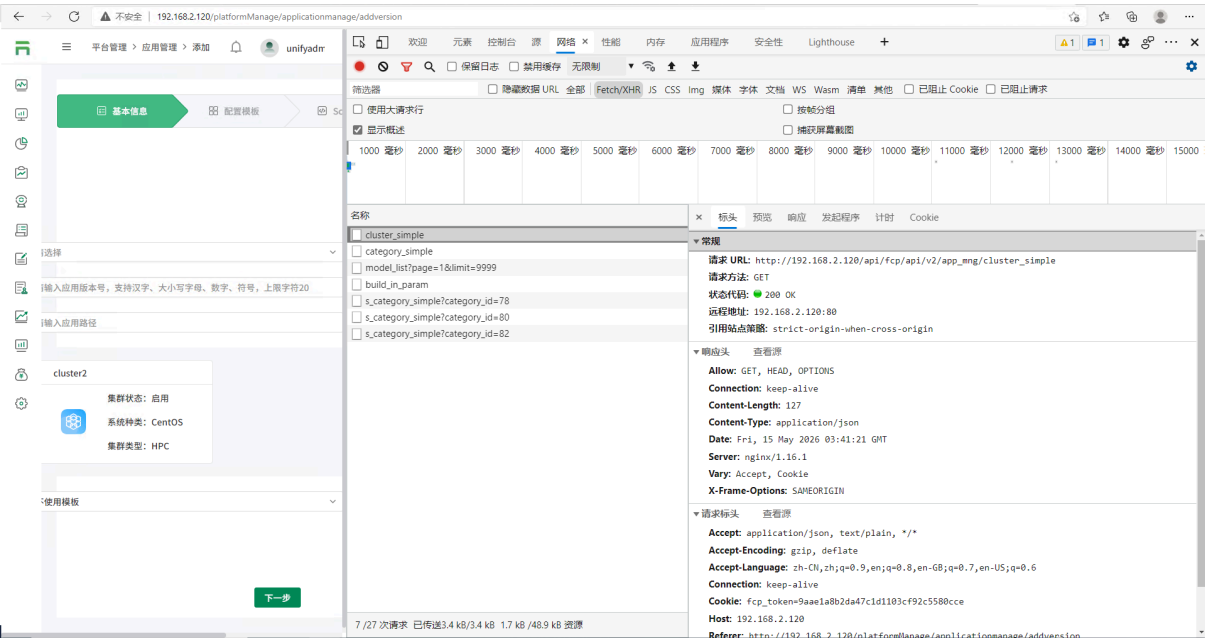
3. 业务场景复盘

在 UI 界面上，它的表现如下：

- 1. 用户操作：在“所属大类”下拉框选择了“编程语言”（假设 ID 是 78）。
- 2. 接口触发：前端发起请求 `GET /.../s_category_simple?category_id=78`。
- 3. 数据回显：后端返回该分类下的“Python”、“Java”、“Go”等具体应用。
- 4. 结果：第二个下拉框（应用名称）被填满，用户可以继续选择。

接口	作用	核心参数	对应 UI
<code>category_simple</code>	加载大类	<code>pid=0</code>	第一个下拉框
<code>s_category_simple</code>	加载小类	<code>pid=[选中的大类 ID]</code>	第二个下拉框（随第一个联动）

八 集群信息查询接口cluster_simple GET



`views.py`

```
class ClusterSimpleView(APIView):
    def get(self, request):
        cluster_list = list(Cluster.objects.exclude(type="AI").values('cluster_id', 'name', 'type', 'os', 'is_active'))
        ret = {
            "code": 0,
            "message": "success",
            "data": {
                "cluster": cluster_list
            }
        }
        return Response(ret)
```

1. 核心功能：获取环境列表

当你准备添加一个应用版本时，系统需要知道这个版本是运行在哪个环境下的。这个接口就是为了在页面上展示那些“集群卡片”（比如你之前看到的 CentOS、HPC 等信息）。

2. 关键代码逻辑

• 数据过滤 (`exclude(type="AI")`):

- 这是一个非常有意思的细节。代码通过 `exclude(type="AI")` 排除了类型为“AI”的集群。
- 业务含义：**这说明该系统将计算资源分成了“普通应用集群”和“AI专项集群”。在目前这个“应用版本管理”模块中，系统只允许你将应用发布到非 AI 类型的集群上。

• 精简字段：

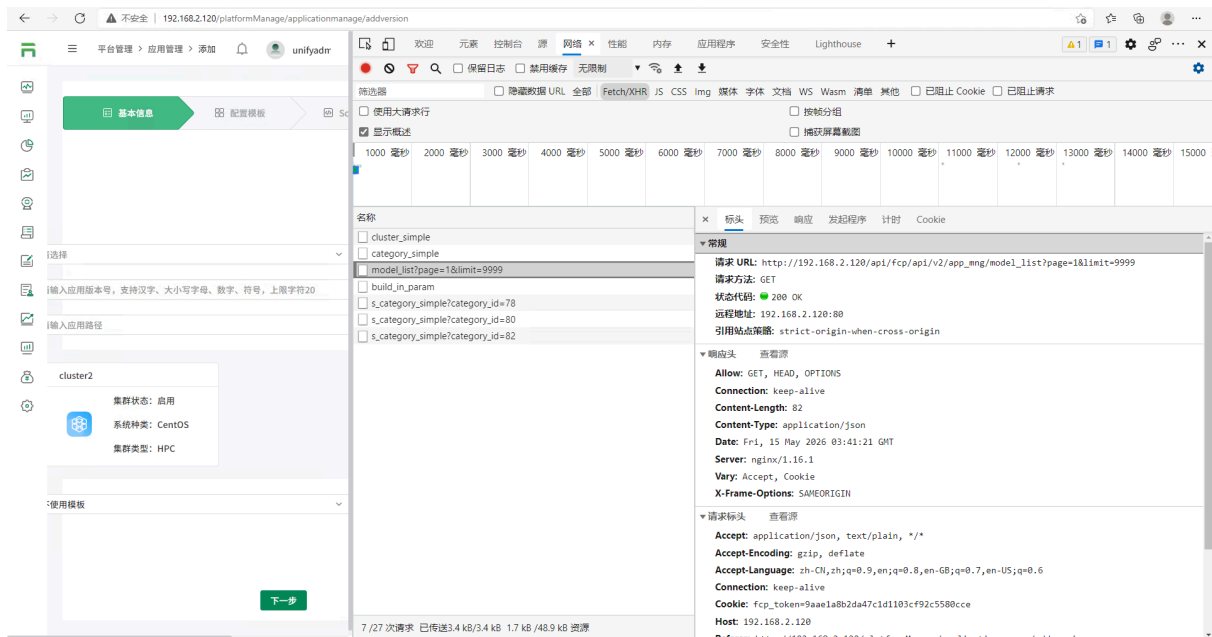
- 它只提取了几个关键信息：`cluster_id`（唯一标识）、`name`（集群名称）、`type`（集群类型，如 HPC）、`os`（操作系统，如 CentOS）、`is_active`（激活状态）。
- 这些信息刚好对应了前端页面卡片上显示的那些标签。

3. 在业务流程中的位置

在“添加应用版本”的第一步（基本信息）中：

1. 页面加载时调用这个接口。
2. 后端返回一组集群数据。
3. 前端根据这些数据生成你看到的“cluster2”这种选择卡片。
4. 用户点击某个卡片，就选定了该版本的运行物理环境。

九 获取系统中存在的模板列表的查询接口model_list GET



views.py

1. 接口基本定义

- 路由: `/api/fcp/api/v2/app_mng/model_list`
- 权限: 需要 `app_mng_get` 权限。
- 核心逻辑类: `AppModel().get_list(request)`。

2. 关键输入参数（筛选器）

```

class AppModel(object):
    def get_list(self, request):
        source = request.GET.get("source", None)
        model_name = request.GET.get("model_name", None)
        s_time = request.GET.get("s_time", None)
        e_time = request.GET.get("e_time", None)

        page_index = request.GET.get("page", 1)
        page_size = request.GET.get("limit", 10)

        if page_index in ["", None]:
            page_index = 1
        if page_size in ["", None]:
            page_size = 10

        page_index = int(page_index)
        page_size = int(page_size)

        data = {
            "page": page_index,
            "limit": page_size,
            "count": 0,
            "model": []
        }

```

你可以通过以下参数来精准查找模型：

- `model_name`：模型名称（支持模糊查询，代码里用了 `contains`）。
- `source`：模型来源。
- `s_time` / `e_time`：模型创建的时间范围（使用了 SQL 的 `BETWEEN` 逻辑）。
- `page` / `limit`：标准的分页参数，默认每页 10 条。

3. 后端处理逻辑流程

```

obj = TbAppModel.objects
if model_name not in ["", None]:
    obj = obj.filter(model_name__contains=model_name)
if source not in ["", None]:
    obj = obj.filter(source=source)
if s_time not in ["", None] and e_time not in ["", None]:
    cond = f"create_time BETWEEN '{s_time}' AND '{e_time}'"
    obj = obj.extra(where=[cond])

val = obj.values("app_model_id", "model_name", "source", "create_time").order_by("-create_time")

```

1. **数据抓取**: 从数据库表 `TbAppModel` 中提取数据。
2. **动态过滤**: 根据你填写的筛选条件，动态拼凑查询语句。
3. **排序**: 默认按 `create_time` **降序排列** (`create_time`)，确保你最新创建的模型排在最前面。

```

.....total_num = len(val)

.....page_num = int(total_num / page_size)
.....else_num = total_num % page_size
.....if else_num > 0:
.....    page_num = page_num + 1

.....if page_index > page_num:
.....    page_index = page_num

.....d_i_s = page_size * (page_index - 1)
.....d_i_e = page_size * page_index

.....if d_i_s < 0:
.....    d_i_s = 0

.....val = val[d_i_s:d_i_e]

```

1. 分页计算:

- 计算总数 (`total_num`) 和总页数 (`page_num`)。
- 如果请求的页码超出了总页数，会自动修正为最后一页，防止返回空数据导致前端报错。

```

    for i in val:
        d = {
            "model_id": i["app_model_id"],
            "model_name": i["model_name"],
            "source": i["source"],
            "create_time": str(i["create_time"])
        }
        data["model"].append(d)

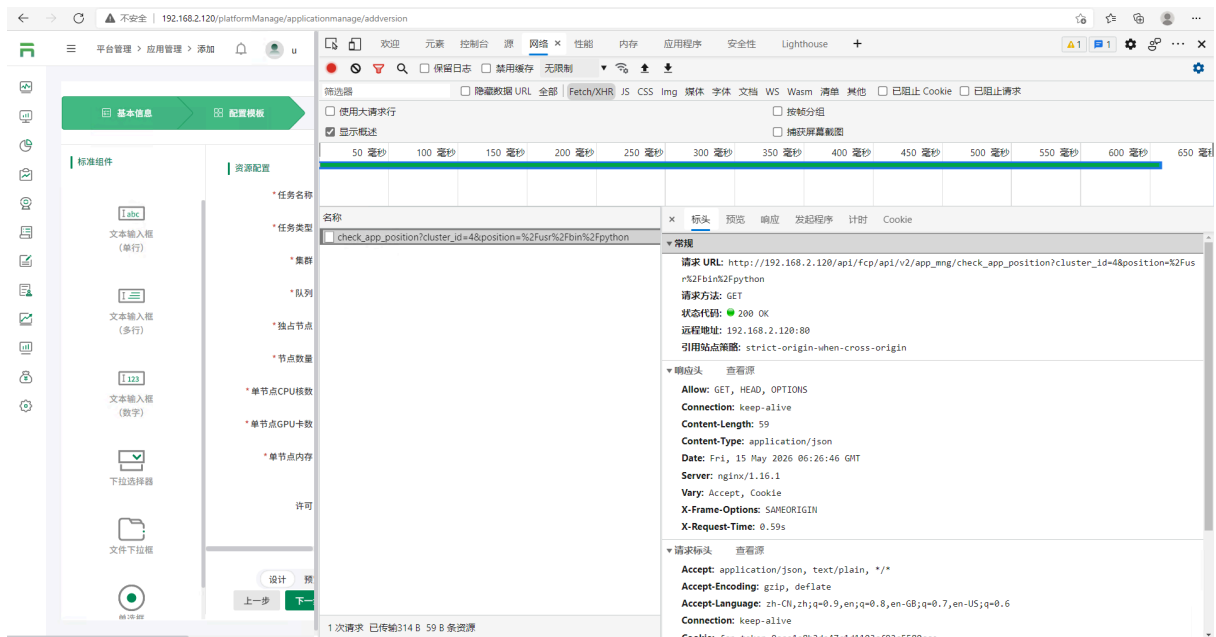
    data["count"] = total_num
    result = {
        "code": 0,
        "message": "success",
        "data": data
    }

    return result

```

1. **数据脱敏:** 接口最终只返回 `model_id`、`model_name`、`source` 和 `create_time`。这属于典型的“概要列表”设计，只给前端展示最必要的标识信息。

十 路径合法性校验接口 check_app_position GET



views.py

```
class CheckAppPositionView(APIView):
    @method_decorator(permission(user_type=["admin"], permission="plat_set_get"))
    def get(self, request):
        r = check_app_position(request)
        return Response(r)
```

1. 关键输入参数

```
def check_app_position(request):
    cluster_id = request.GET.get("cluster_id", None)
    app_path = request.GET.get("position", None)
    app_version_id = request.GET.get("app_version_id", None)
    result = {
        "code": -1,
        "message": "failed"
    }

    # if cluster_id in ["", None]:
    #     result["err"] = "请指定集群id"
    #     return result
    if app_path in ["", None]:
        result["err"] = "请指定应用路径"
        return result
```

- `cluster_id`: 你选中的那个集群 ID (即你要在哪台机器上跑这个应用)。
- `position`: 你填写的应用路径 (比如 `/usr/bin/python`)。

2. 后端核心处理逻辑

这是该系统非常硬核的一个地方，它不仅仅是检查数据库，而是真的去执行了系统指令：

```
flag = True
cmd = f"test -e {app_path}"

if app_version_id not in ["", None]:
    cluster_id_list = AppCluster.objects.filter(app_version_id=app_version_id).values_list('cluster_id', flat=True)
else:
    cluster_id_list = cluster_id.strip().split(",")
for c_id in cluster_id_list:
    obj = TbCluster.objects.filter(cluster_id=c_id)
    if obj.count() > 0:
        try:
            ret, out = Base().cluster_cmd(cluster_id=c_id, cmd=cmd)
            if ret != 0:
                flag = False
                break
        except:
            flag = False
    else:
        flag = False
        break
```

1. 构造 Linux 指令：后端会生成一条 shell 命令：`test -e {app_path}`。

`test -e` 是 Linux 中用来检测“文件或目录是否存在”的标准指令。

2. 远程执行：通过 `Base().cluster_cmd(...)`，系统会将这条指令发送到你选中的远程集群节点上去执行。


```

... if flag:
...     result = {
...         "code": 0,
...         "message": "success",
...         "data": {
...             "check_result": "ok"
...         }
...     }
... else:
...     result = {
...         "code": 0,
...         "message": "success",
...         "data": {
...             "check_result": "invalid"
...         }
...     }
... return result

```

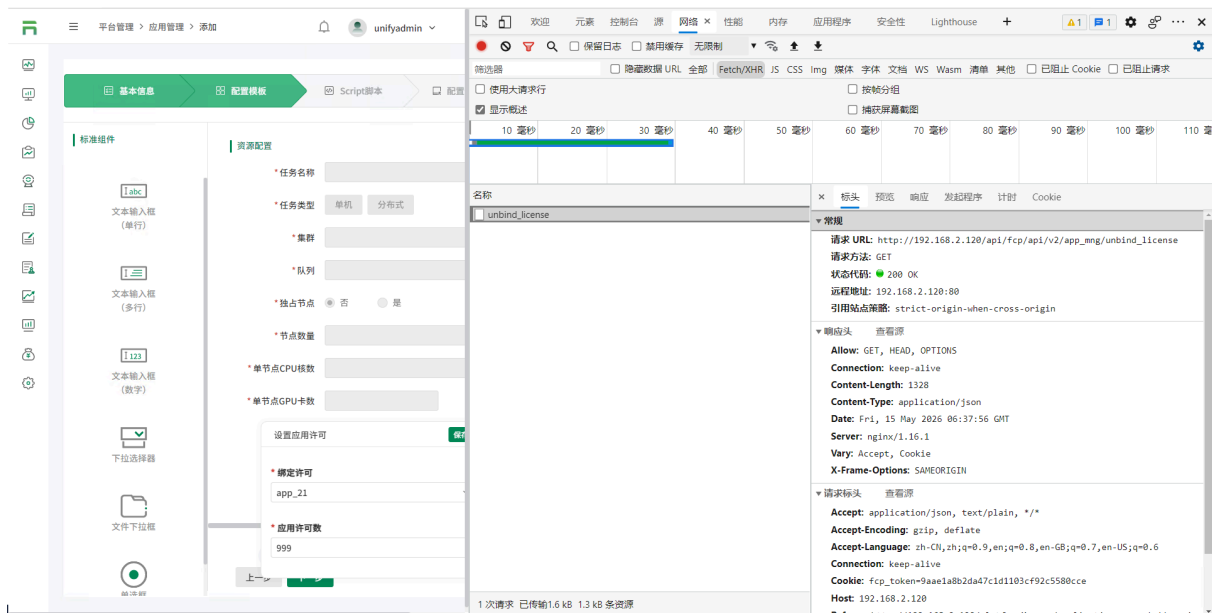
1. 结果判定：

- 如果指令返回码为 `0`（Linux 惯例，0 代表成功），说明路径**真实存在**，返回 `ok`。
- 如果返回码不是 `0`，说明路径**不存在**或者服务器无法访问该路径，返回 `invalid`。

3. 业务逻辑亮点

- **多集群兼容**：代码中有一个 `for c_id in cluster_id_list` 的循环。这意味着如果你的应用版本关联了多个集群，它会去**每一个**集群上都跑一遍检查。只要有一个集群上找不到这个路径，它就会判定为无效。
- **实时反馈**：这是一个典型的“所见即所得”校验。你一边填，它一边查，不用等到最后提交时才告诉你路径错了，用户体验很好。

十一 未绑定许可查询接口unbind_license GET



views.py

```
class UnbindLicenseView(APIView):
    @method_decorator(permission(user_type=["admin"], permission="plat_set_get"))
    def get(self, request):
        r = License().get_unbind(request)
        return Response(r)
```

1. 核心功能：获取可用许可池

系统中的许可证（License）通常是有限的资源。一个许可证如果已经绑定给了某个应用版本，就不能再给另一个用。这个接口的作用就是过滤掉那些“已婚”的许可，只展示“单身”（未绑定）的许可。

2. 关键代码逻辑

```
class License(object):
    def get_unbind(self, request):
        data = {"unbind_license": []}

        obj = TbAppLicense.objects.filter(is_bind=0)
```

- 数据过滤 (`is_bind=0`):

- 这是接口的“灵魂”逻辑。它去 `TbAppLicense` 表中查询，只抓取 `is_bind=0` 的记录。
- `is_bind=0` 代表该许可目前处于未绑定/空闲状态。

- 提取关键信息：

```
..... val = obj.values("app_license_id", "lic_name_map")
..... for i in val:
.....     app_license_id = i["app_license_id"]
.....     lic_name_map = i["lic_name_map"]

.....     d = {
.....         "app_license_id": app_license_id,
.....         "lic_name_map": lic_name_map
.....     }
.....     data["unbind_license"].append(d)

..... result = {
.....     "code": 0,
.....     "message": "success",
.....     "data": data
..... }

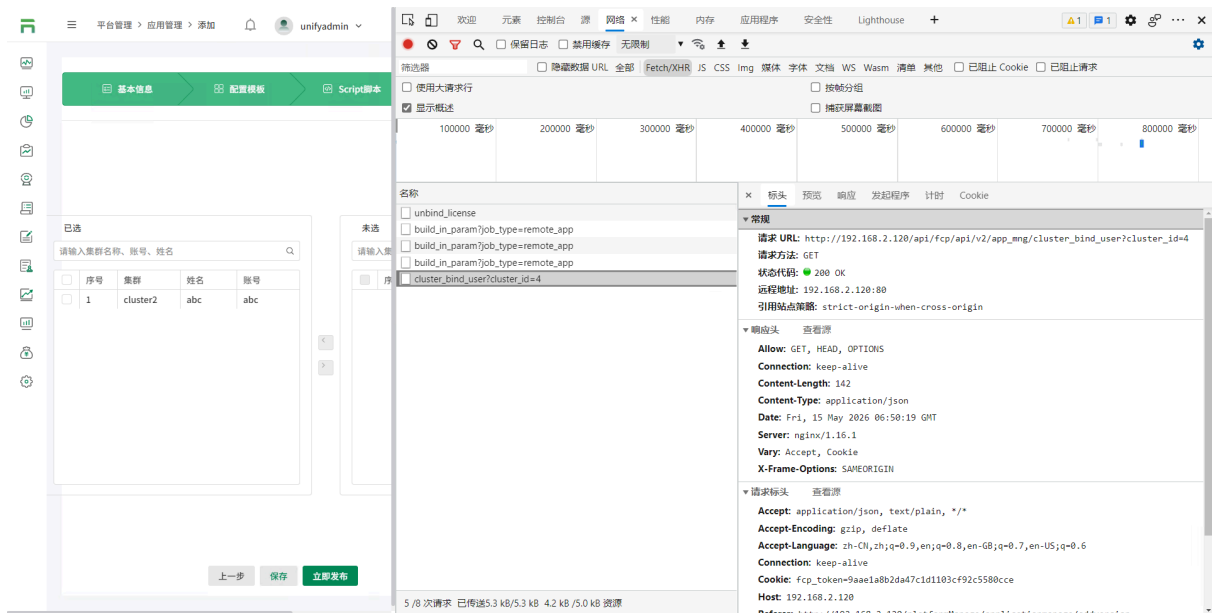
..... return result
```

- 它只提取了 `app_license_id`（许可 ID）和 `lic_name_map`（许可名称映射/描述）。
- 这些信息足够前端生成一个下拉列表，让管理员通过名称来选择合适的许可。

3. 业务意义

- **防止冲突：**通过只展示 `is_bind=0` 的数据，从源头上避免了将同一个许可证重复分配给多个应用的问题。
- **资源清点：**它让管理员清楚地看到目前手里还有多少“余粮”（可用许可）。

十二 集群关联用户查询接口cluster_bind_user GET



views.py

```
157 class ClusterBindUserView(APIView):
158     @method_decorator(permission(user_type=["admin"], permission="plat_set_get"))
159     def get(self, request):
160         r = ClusterBindUser().get(request)
161         return Response(r)
162
```

1. 核心功能：账号匹配

在高性能计算（HPC）或集群环境中，应用通常需要一个具体的系统账号（User Account）来运行。这个接口的作用是：找出哪些用户在选定的集群上有账号，并准备好给他们授权。

2. 后端核心逻辑拆解

这段代码逻辑比较绕，因为它涉及到了三张表的“大合体”：

```

48     def get(self, request):
49         cluster_id = request.GET.get("cluster_id", None)
50         exclude_app_version_id = request.GET.get("exclude_app_version_id", None)
51         result = {
52             "code": -1,
53             "message": "failed"
54         }
55
56         if cluster_id in ["", None]:
57             result["err"] = "请指定集群id"
58             return result
59
60         cluster_id_list = cluster_id.strip().split(",")
61         obj = TbUserCluster.objects.filter(cluster_id__in=cluster_id_list)
62         if exclude_app_version_id not in ["", None]:
63             user_id_list = UserApp.objects.filter(app_version_id=exclude_app_version_id,
64                                                     cluster_id=cluster_id).values_list("user_id", flat=True)
65             obj = obj.filter(cluster_id__in=cluster_id_list).exclude(user_id__in=user_id_list)
66             val = obj.values("user_id", "cluster_id")
67             user_id_list = []
68             cluster_id_list = []
69             uid_cid_dict = {}
70             for k in val:
71                 uid = k["user_id"]
72                 cid = k["cluster_id"]
73                 user_id_list.append(uid)
74                 cluster_id_list.append(cid)
75
76                 if uid not in uid_cid_dict.keys():
77                     uid_cid_dict[uid] = []
78                 uid_cid_dict[uid].append(cid)
79

```

第一步：锁定范围

1. **接收参数**：拿到 `cluster_id`（目标集群）和 `exclude_app_version_id`（要排除的应用版本 ID）。
2. **初步筛选**：从 `TbUserCluster`（用户-集群关联表）中，找出所有在该集群下有权限的用户。
3. **去重逻辑 (关键)**：通过 `exclude` 过滤掉那些已经绑定过该应用版本的用户。

目的：防止你把同一个应用给同一个用户授权两次。系统只给你展示“还没领到证”的用户。

```

80         c_obj = TbCluster.objects.filter(cluster_id__in=cluster_id_list)
81         c_val = c_obj.values("cluster_id", "name")
82         clu_dict = {}
83         for c in c_val:
84             cid = c["cluster_id"]
85             n = c["name"]
86             clu_dict[cid] = n
87
88         u_obj = TbUser.objects.filter(user_id__in=user_id_list)
89         u_val = u_obj.values("user_id", "username", "fullname")

```

第二步：组装详细信息

因为关联表里只有 ID（一堆数字），为了让管理员在网页上能看懂，后端又做了两次“查表进修”：

1. **查集群名**：去 `TbCluster` 表里把 ID 换成好听的集群名字（如 "cluster2"）。
2. **查用户名**：去 `TbUser` 表里把用户 ID 换成真实的 `username` 和 `fullname`（真实姓名）。

```
91         bind_user = []
92         for i in u_val:
93             uid = i["user_id"]
94             if uid in uid_cid_dict.keys():
95                 cid_list = uid_cid_dict[uid]
96                 for cid in cid_list:
97                     d = {
98                         "cluster_id": cid,
99                         "cluster_name": clu_dict.get(cid, None),
100                        "user_id": uid,
101                        "username": i["username"],
102                        "fullname": i["fullname"]
103                    }
104                 bind_user.append(d)
105         result = {
106             "code": 0,
107             "message": "success",
108             "data": {"bind_user": bind_user}
109         }
110         return result
```

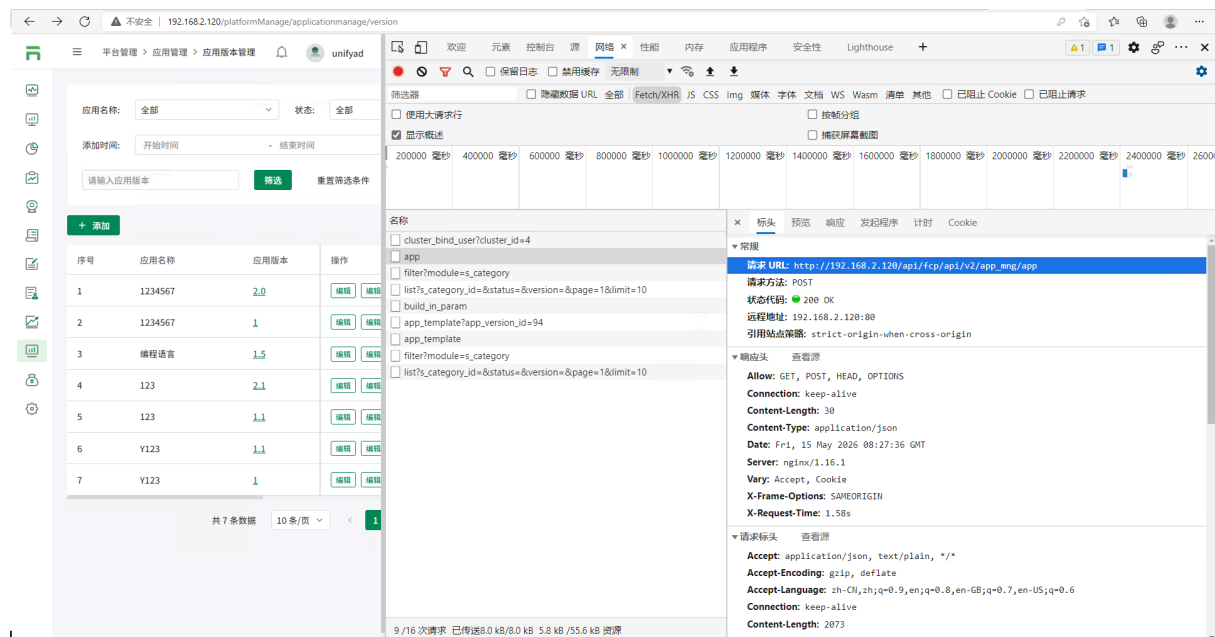
第三步：打包返回

最终返回一个结构化的列表，每个条目都包含：**集群名 + 用户名 + 姓名**。这正好对应了左侧表格里的列。

3. 业务场景复盘

1. **管理员操作**：进入“授权”步骤。
2. **接口触发**：前端自动根据第一步选的集群 ID 发起 `GET /.../cluster_bind_user?cluster_id=4`。
3. **页面展示**：右侧列表刷出了一堆账号。
4. **最终授权**：管理员把右侧的账号勾选到左侧，这就完成了“谁在哪个集群上可以用这个软件”的配置。

十三 新增/获取应用版本app



views.py

```
class AppView(APIView):
    @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
    def post(self, request):
        r, event = App().post(request)
        # 记录日志
        if r["code"] == 0 and event not in ["", None]:
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                       operate_type="新增应用版本",
                                       operate_content=event, state=True)
        return Response(r)

    @method_decorator(permission(usertype=["admin"], permission="plat_set_get"))
    def get(self, request):
        r = App().get(request)
        return Response(r)
```

新增应用POST

1. 严格的“资格审查” (Validation)

在动任何数据之前，代码先进行了密集的校验：

```

22 class App(object):
23     def post(self, request):
24         category_id = request.POST.get("s_category_id", None)
25         version = request.POST.get("version", None)
26         app_path = request.POST.get("position", None)
27         cluster_id = request.POST.get("cluster_id", None)
28         param_data = request.POST.get("param_data", None)
29         script_exec = request.POST.get("script_exec", None)
30         is_remote = request.POST.get("is_remote", None)
31         script_remote = request.POST.get("script_remote", None)
32         bind_user_id = request.POST.get("bind_user", None)
33         status = request.POST.get("status", None)
34
35         bind_lic_id = request.POST.get("app_license_id", None)
36         bind_lic_num = request.POST.get("bind_lic_num", None)
37
38         event = None
39         result = {
40             "code": -1,
41             "message": "failed"
42         }
43
44         if category_id in ["", None]:
45             result["err"] = "请指定应用id"
46             return result, event
47         if version in ["", None]:
48             result["err"] = "请指定应用版本"
49             return result, event
50         if app_path in ["", None]:
51             result["err"] = "请指定应用位置"
52             return result, event
53         if cluster_id in ["", None]:
54             result["err"] = "请指定集群id"

```



```

55         return result, event
56     if param_data in ["", None]:
57         result["err"] = "请指定应用参数"
58         return result, event
59     if script_exec in ["", None]:
60         result["err"] = "请指定Script脚本"
61         return result, event
62
63     if is_remote not in [0, "0", 1, "1"]:
64         result["err"] = "请指定交互应用状态"
65         return result, event
66     if script_remote in ["", None]:
67         script_remote = ""
68
69     if status in ["", None]:
70         result["err"] = "请确定发布状态"
71         return result, event
72     if bind_lic_id in ["", None]:
73         result["err"] = "请指定绑定许可id"
74         return result, event
75     if bind_lic_num in ["", None]:
76         result["err"] = "请指定应用许可数"
77         return result, event
78     jud, ret = self.check_param(param_data)
79     if jud != 0:
80         return ret, event

```

```

82         bind_lic_num = int(bind_lic_num)
83
84         if str(status) == "1":
85             status_data, id_data = AppBelongsStatus().s_p_category(category_id)
86             result = {
87                 "code": 0,
88                 "message": "success",
89                 "data": status_data
90             }
91
92         return result, event
93
94
95         obj_num = TbAppVersion.objects.filter(category_id=category_id, version=version).count()
96         if obj_num > 0:
97             result["err"] = "应用模板已存在"
98             return result, event
99
100         # 查询应用名
101         ac_obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)
102         ac_val = ac_obj.values("category_name")
103         if len(ac_val) > 0:
104             category_name = ac_val[0]["category_name"]
105         else:
106             result["err"] = "未查询到指定应用"
107             return result, event
108
109         # 查询许可
110         al_obj = TbAppLicense.objects.filter(app_license_id=bind_lic_id, is_bind=0)
111         if al_obj.count() == 0:
112             result["err"] = "未查询到指定许可"
113             return result, event
114
App > post()

```

- **必填校验：**版本号、执行路径、集群、脚本、许可 ID 等 10 多个参数一个都不能少。
- **重复检查：**去 `TbAppVersion` 查一下，同一个应用下是否已经存在这个版本号，防止冲突。
- **参数合法性：**通过 `check_param` 检查你填写的那些运行参数格式对不对。

2. 硬核的“远程核销” (Remote License Check)

这是该接口最体现“高性能计算（HPC）”背景的地方：

```

116         cluster_id_list = cluster_id.strip().split(",")
117
118         # 判断集群是否存在许可信息
119         lic_name_map = al_obj.values("lic_name_map")[0]["lic_name_map"]
120         # cmd = f"scontrol show license {lic_name_map} -o"
121         cmd = f"scontrol show license {lic_name_map}@{LIC_SERVER} -o"
122         for c_id in cluster_id_list:
123             ret, out = Base().cluster_cmd(cluster_id=c_id, cmd=cmd)
124             out = out.strip()
125             if int(ret) != 0 or len(out) == 0 or out == "No licenses configured in Slurm.":
126                 result["err"] = "集群未配置指定许可"
127                 result["out"] = f"{out}"
128                 result["ret"] = f"{ret}"
129             return result, event
130
131         # 每个集群绑定许可的数量
132         every_clu_lic_num = int(bind_lic_num / len(cluster_id_list))
133
134         ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
135
136         bind_user_id_list = []
137         if bind_user_id not in ["", None]:
138             # bind_user_id_list = bind_user_id.strip().split(",")
139             bind_user_id_list = eval(bind_user_id)

```

- **实地考察：**后端会构造一条 Linux 指令：`scontrol show license ...`。
- **跨机执行：**它会登录到你选的那个集群上，实地检查那个集群是否真的配置了你选的这个许可证。
- **一票否决：**如果集群上没配置对应的 License 资源，接口会直接报错返回，不允许添加。

3. 开启“原子事务” (Atomic Transaction)

```

141         with transaction.atomic():
142             if str(status) == "2":
143                 status_data, pid = AppBelongsStatus().s_p_category(category_id)
144                 data_u_cty = {
145                     "status": 1,
146                     "update_time": ctime
147                 }
148                 if str(status_data["category"]["status"]) == "0":
149                     TbAppCategory.objects.filter(is_extern=1, app_category_id=pid).update(**data_u_cty)
150                 if str(status_data["s_category"]["status"]) == "0":
151                     TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id).update(**data_u_cty)
152             status = 1
153
154             # 基本信息
155             data_c_ver = {
156                 "category_id": category_id,
157                 "version": version,
158                 "app_path": app_path,
159                 "status": status,
160                 "limit_type": "cpu",
161                 "create_time": ctime,
162                 "update_time": ctime
163             }
164             # 保存基本信息
165             app_version_id = TbAppVersion.objects.create(**data_c_ver).app_version_id

```

```

168         script_exec = script_exec.replace("\r\n", "\n")
169         script_remote = script_remote.replace("\r\n", "\n")
170         # 保存模板信息
171         data_c_template = {
172             "app_version_id": app_version_id,
173             "param_data": param_data,
174             "script_exec": script_exec,
175             "is_remote": is_remote,
176             "script_remote": script_remote,
177             "create_time": ctime,
178             "update_time": ctime
179         }
180         TbAppTemplate.objects.create(**data_c_template)
181
182         for c_id in cluster_id_list:
183             # 绑定集群信息
184             data_c_cluster = {
185                 "app_version_id": app_version_id,
186                 "cluster_id": c_id
187             }
188             TbAppCluster.objects.create(**data_c_cluster)
189
190             #
191             cmd_c = f"sacctmgr update resource name={lic_name_map} " \
192             f"server={LIC_SERVER} set count={every_clu_lic_num} -i"
193             ret_c, out_c = Base().cluster_cmd(cluster_id=c_id, cmd=cmd_c)
194             if ret_c != 0:
195                 result["err"] = "设置集群许可数量失败"
196                 result["err2"] = out_c.strip().split("\n")
197             return result, event

```

```

200 ... # 绑定许可信息
201 ... data_c_lic_clu = {
202 ...     "cluster_id": c_id,
203 ...     "app_license_id": bind_lic_id,
204 ...     "lic_num": every_clu_lic_num
205 ... }
206 ... TbAppLicenseCluster.objects.create(**data_c_lic_clu)

```

由于接下来的操作要写入 5-6 张表，代码使用了 `with transaction.atomic()`。

这意味着：如果最后一张表写入失败，前面写进去的所有数据都会自动撤回（回滚），保证数据库里绝对不会出现“半成品”数据。

4. 级联发布逻辑（向上打通）

我们在之前“发布版本”接口里见过的逻辑在这里也有体现：

- **自动开路：**如果你在添加版本时选择了“立即发布”，系统会检查该应用所属的“小类”和“大类”。如果它们目前是隐藏状态，系统会强制把大类和小类全部发布，确保用户能立刻看到新上的版本。

5. 数据的“天女散花” (Data Distribution)

```

208 ... if len(bind_user_id_list) > 0:
209 ...     # 绑定用户信息
210 ...     ua_data = []
211 ...     for uc_id in bind_user_id_list:
212 ...         u_id = uc_id["user_id"]
213 ...         clu_id = uc_id["cluster_id"]
214 ...         ua_d = TbUserApp(user_id=u_id, cluster_id=clu_id, app_version_id=app_version_id, create_time=ctime)
215 ...         ua_data.append(ua_d)
216 ...     TbUserApp.objects.bulk_create(ua_data)
217 ...
218 ...     # 更新绑定许可状态
219 ...     lic_name = f"{category_name}_{version}"
220 ...     data_u_lic = {
221 ...         "lic_name": lic_name,
222 ...         "is_bind": 1,
223 ...         "app_version_id": app_version_id,
224 ...         "lic_num": bind_lic_num,
225 ...         "update_time": ctime
226 ...     }
227 ...     al_obj.update(**data_u_lic)
228 ...
229 ...     # event = f"应用名称: {category_name}; 应用版本: {version}"
230 ...     event = '''<p class="log log_obj"><span class="obj">应用名称:
231 ...     </span><span class="val">%s</span>; <span class="obj">应用版本: </span><span class="val">%s</span></p>''' \
232 ...     % (category_name, version)
233 ...
234 ...     result = {
235 ...         "code": 0,
236 ...         "message": "success"
237 ...     }
238 ...
239 ...     return result, event

```

最后，它把你填写的这大堆信息拆分并存入不同的篮子：

1. **TbAppVersion**：存版本号、路径、状态。
2. **TbAppTemplate**：存那些复杂的运行脚本和参数。
3. **TbAppCluster**：记录这个版本跑在哪个集群。
4. **TbAppLicenseCluster**：**核心逻辑**。执行 **sacctmgr** 命令，在 Linux 系统底层给这个应用分配授权额度，并记录在案。
5. **TbUserApp**：批量绑定那些被授权使用该应用的用户。
6. **TbAppLicense**：把许可证标记为“已绑定”，从此这个证就名花有主了。

```
141         with transaction.atomic():
142             if str(status) == "2":
143                 status_data, pid = AppBelongsStatus().s_p_category(category_id)
144                 data_u_cty = {
145                     "status": 1,
146                     "update_time": ctime
147                 }
148                 if str(status_data["category"]["status"]) == "0":
149                     TbAppCategory.objects.filter(is_extern=1, app_category_id=pid).update(**data_u_cty)
150                 if str(status_data["s_category"]["status"]) == "0":
151                     TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id).update(**data_u_cty)
152
153                 status = 1
154
155                 # 基本信息
156                 data_c_ver = {
157                     "category_id": category_id,
```

问题？这个函数什么意思

```

6 class AppBelongsStatus(object):
7     def s_p_category(self, c_id):
8         obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=c_id)
9         val = obj.values("category_name", "status", "pid")
10
11         data = {
12             "category": {
13                 "name": None,
14                 "status": None
15             },
16             "s_category": {
17                 "name": None,
18                 "status": None
19             }
20         }
21
22         pid = None
23         if len(val) > 0:
24             s_category_name = val[0]["category_name"]
25             s_status = val[0]["status"]
26
27             data["s_category"]["name"] = s_category_name
28             data["s_category"]["status"] = s_status
29
30             pid = val[0]["pid"]
31
32             p_obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=pid)
33             p_val = p_obj.values("category_name", "status")
34
35             if len(p_val) > 0:
36                 p_category_name = p_val[0]["category_name"]
37                 p_status = p_val[0]["status"]
38
39                 data["category"]["name"] = p_category_name
40                 data["category"]["status"] = p_status
41
42         return data, pid
43

```

• 第一步：查子类（小应用）

- 它拿着你传进去的 `c_id` 去数据库里找。
- 它会记录下这个应用的名称、当前状态，以及最重要的——它的父 ID (`pid`)。

• 第二步：查父类（大类）

- 通过刚才拿到的 `pid`，它再次去数据库里找那个“大类”。
- 它会记录下这个大类的名称和当前状态。

• 第三步：打包汇报

- 它把查到的所有信息封装进一个叫 `data` 的字典里，并把 `pid` 也一起返回。

解决了：

1. 调用 `s_p_category` 查出这一条线上的所有状态。
2. 如果发现父类是关着的（`status=0`），系统会自动执行 `update(status=1)` 把它**强行打开**。
3. 如果发现子类也是关着的，同样**强行打开**。

获取应用GET

1. 核心功能：应用详情与集群状态查询

```
241     def get(self, request):
242         app_version_id = request.GET.get("app_version_id", None)
243
244         result = {
245             "code": -1,
246             "message": "failed"
247         }
248
249         if app_version_id in ["", None]:
250             result["err"] = "请指定应用版本id"
251         return result
```

该接口的作用是：当管理员或用户选中一个具体的**应用版本**（`app_version_id`）时，系统需要展示该版本的基本信息（如名称、图标、路径），并列出当前环境中所有可用的集群，同时标明该应用**已经部署（绑定）到了哪些集群上**。

2. 后端核心逻辑拆解

代码逻辑主要分为三个阶段，实现了从“基础数据”到“关联状态”的完整组装：

第一步：获取应用版本及分类基础信息

- **参数校验**：首先检查 `app_version_id` 是否为空，若为空则直接拦截并报错。

```
obj_ver = TbAppVersion.objects.filter(is_extern=1, app_version_id=app_version_id)
val_ver = obj_ver.values("category_id", "version", "app_path", "create_time")

if len(val_ver) > 0:
    category_id = val_ver[0]["category_id"]
    obj_cty = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)
    val_cty = obj_cty.values("category_name", "icon_name")
```

- **版本定位**：从 `TbAppVersion` 表中筛选出对应的记录，获取该版本的核心字段（分类 ID、版本号、应用路径等）。
- **分类关联**：利用获取到的 `category_id` 去 `TbAppCategory` 表中“溯源”，查询该应用所属的类别名称（`app_name`）和图标名称（`icon_name`）。

第二步：数据脱敏与安全处理

- **地址加密：**在组装 `base_data` 时，代码对 `icon_url` 进行了 `Password().encrypt()` 加密处理。

目的：隐藏服务器内部真实的存储路径或私有资源地址，防止前端直接暴露敏感的资源 URL，增强了安全性。

第三步：集群绑定状态的“十字交叉”对比（核心难点）

这是代码中最巧妙的部分，它处理了“已绑定”和“未绑定”的逻辑：

```
256     if len(val_ver) > 0:
257         category_id = val_ver[0]["category_id"]
258         obj_cty = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)
259         val_cty = obj_cty.values("category_name", "icon_name")
260
261     if len(val_cty) > 0:
262         app_name = val_cty[0]["category_name"]
263         icon_name = val_cty[0]["icon_name"]
264         icon_url = f"{PRJ_IMG_APP_PATH}/{category_id}/{icon_name}"
265
266         base_data = {
267             "icon_url": Password().encrypt(icon_url),
268             "app_name": app_name,
269             "version": val_ver[0]["version"],
270             "app_path": val_ver[0]["app_path"],
271             "create_time": str(val_ver[0]["create_time"])
272         }
273
274         obj_clu = TbAppCluster.objects.filter(app_version_id=app_version_id)
275         val_clu = obj_clu.values("cluster_id")
276
277         bind_clu_id_list = []
278         for i in val_clu:
279             bind_clu_id_list.append(i["cluster_id"])
280
281         clu_val = TbCluster.objects.filter(is_active=1).exclude(type="AI").values("cluster_id", "name", "type", "os", "is_active")
282
283         cluster_data = []
284         for c_i in clu_val:
```

```
285             cluster_id = c_i["cluster_id"]
286             is_bind = 0
287             if cluster_id in bind_clu_id_list:
288                 is_bind = 1
289
290             c_data = {
291                 "cluster_id": cluster_id,
292                 "name": c_i["name"],
293                 "is_bind": is_bind,
294                 "type": c_i["type"],
295                 "os": c_i["os"],
296                 "is_active": c_i["is_active"]
297             }
298
299             cluster_data.append(c_data)
300
```

```

301 .....data = {
302 .....    "base": base_data,
303 .....    "cluster": cluster_data
304 .....}
305
306 .....result = {
307 .....    "code": 0,
308 .....    "message": "success",
309 .....    "data": data
310 .....}
311 .....else:
312 .....    result = {
313 .....        "code": -1,
314 .....        "message": "failed",
315 .....        "err": "应用不存在"
316 .....    }
317 .....else:
318 .....    result = {
319 .....        "code": -1,
320 .....        "message": "failed",
321 .....        "err": "应用版本不存在"
322 .....    }
323
324 .....return result

```

1. **查已选：**去 `TbAppCluster` 表里搜一下，看看当前这个版本已经和哪些集群 ID 绑定了，存入 `bind_clu_id_list`。
2. **查所有：**从 `TbCluster` 表里取出所有处于激活状态（`is_active=1`）且非 AI 类型的集群。
3. **打标签 (Mapping)：**遍历所有激活的集群，通过 `if cluster_id in bind_clu_id_list` 判断。
 - 如果在已选列表里，`is_bind` 设为 `1`；
 - 如果不在，`is_bind` 设为 `0`。

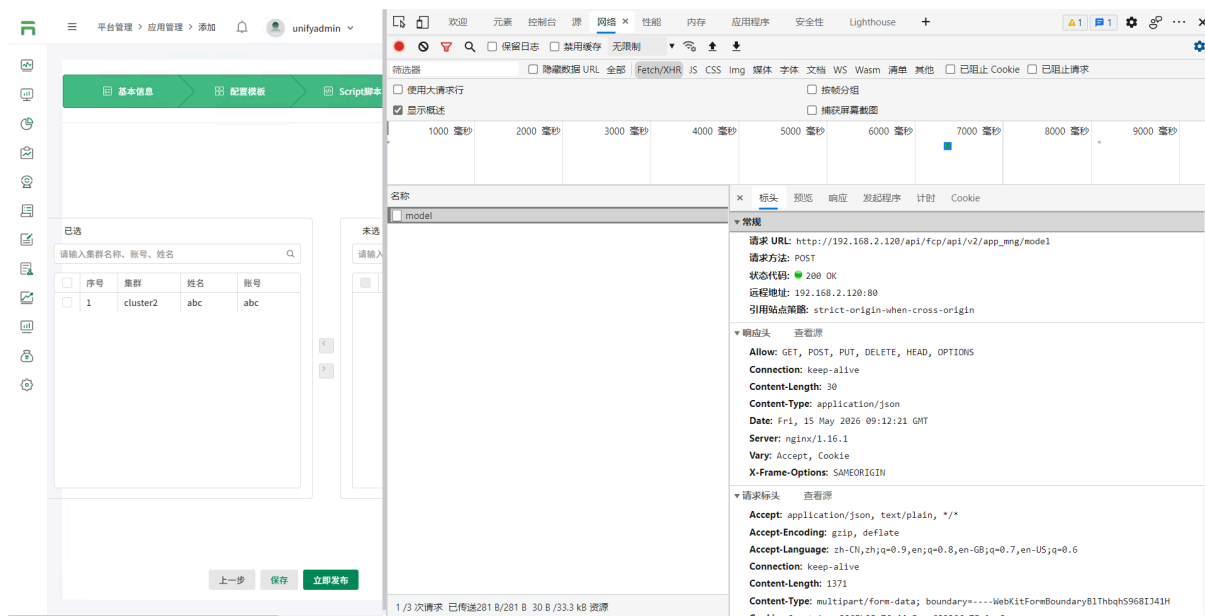
3. 业务场景复盘

- **管理员视图：**当管理员打开“应用分发”或“版本管理”页面，点击某个版本的“分发设置”时。
- **接口触发：**前端发起请求 `GET /.../?app_version_id=123`。

- **页面渲染：**
 - **上方/左侧：**显示应用的图标、版本号（如 v2.1.0）、创建时间。
 - **列表区：**展示一个集群列表。管理员能一眼看到“集群 A（已绑定）”、“集群 B（未绑定）”。
- **最终操作：**管理员可以根据这个列表的 `is_bind` 状态，继续勾选或取消勾选，从而决定将该应用部署到哪些计算节点上。

第三类：模板管理

一 model 四个请求



views.py

```

class ModelView(APIView):
    @method_decorator(permission(usertype=["admin"], permission="app_mng_op"))
    def post(self, request):
        username = TokenOp().get_username(request)
        r = AppModel().post(request, username)
        return Response(r)

    @method_decorator(permission(usertype=["admin"], permission="app_mng_op"))
    def delete(self, request):
        r = AppModel().delete(request)
        return Response(r)

    @method_decorator(permission(usertype=["admin"], permission="app_mng_op"))
    def put(self, request):
        r = AppModel().put(request)
        return Response(r)

    @method_decorator(permission(usertype=["admin"], permission="app_mng_get"))
    def get(self, request):
        r = AppModel().get(request)
        return Response(r)

```

一、GET 接口：获取模型模板详情

1. 核心功能：模板参数调取

该接口的作用是：根据唯一的 `model_id` 获取某个应用模型的详细配置信息。它是前端进入“编辑模式”或“查看详情”时的首选数据源。

2. 后端核心逻辑拆解

```

def get(self, request):
    model_id = request.GET.get("model_id", None)

    result = {
        "code": -1,
        "message": "failed"
    }

    if model_id in ["", None]:
        result["err"] = "请指定模板ID"
        return result

    obj = TbAppModel.objects.filter(app_model_id=model_id)
    if obj.count() == 0:
        result["err"] = "模板不存在"
        return result

```

- **ID 校验：**首先确认请求中是否携带了 `model_id`，避免非法查询。

```

94     val = obj.values("app_model_id", "model_name", "param_data", "script_exec",
95                     "is_remote", "script_remote", "source", "create_time")
96     data = {
97         "model": {
98             "model_id": val[0]["app_model_id"],
99             "model_name": val[0]["model_name"],
100            "param_data": val[0]["param_data"],
101            "script_exec": val[0]["script_exec"],
102            "is_remote": val[0]["is_remote"],
103            "script_remote": val[0]["script_remote"],
104            "source": val[0]["source"],
105            "create_time": str(val[0]["create_time"])
106        }
107    }
108    result = {
109        "code": 0,
110        "message": "success",
111        "data": data
112    }
113
114    return result

```

- **存在性检查：**在 `TbAppModel` 表中进行过滤，如果找不到对应记录，直接返回“模板不存在”的错误，防止后续空对象操作。
- **数据结构组装：**

- 从数据库中提取 `param_data`（应用模板参数）、`script_exec`（执行脚本）等核心字段。
- **时间格式化**：将数据库中的 `create_time` 对象转换为标准字符串，确保前端展示的一致性。

3. 业务场景复盘

- **场景**：管理员在模型列表中点击“修改”按钮。
- **流程**：前端获取 ID 后调用此接口，将返回的脚本内容和参数填入页面的输入框中，供管理员二次编辑。

二、POST 接口：创建新模型模板

1. 核心功能：新模板入库

该接口的作用是：允许管理员定义一个新的应用运行逻辑，包括脚本内容、交互状态（是否远程）以及参数定义。

2. 后端核心逻辑拆解

```

116     def post(self, request, user):
117         model_name = request.POST.get("model_name", None)
118         param_data = request.POST.get("param_data", None)
119         script_exec = request.POST.get("script_exec", None)
120         is_remote = request.POST.get("is_remote", None)
121         script_remote = request.POST.get("script_remote", None)
122
123         result = {
124             "code": -1,
125             "message": "failed"
126         }
127
128         if model_name in ["", None]:
129             result["err"] = "请指定模板名称"
130             return result
131         if param_data in ["", None]:
132             result["err"] = "请指定应用模板"
133             return result
134         if script_exec in ["", None]:
135             result["err"] = "请指定脚本"
136             return result
137         if is_remote not in [0, "0", 1, "1"]:
138             result["err"] = "请指定交互应用状态"
139             return result
140
141         obj = TbAppModel.objects.filter(model_name=model_name)
142         if obj.count() > 0:
143             result["err"] = "模板已存在"
144             return result
145         ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
146         script_exec = script_exec.replace("\r\n", "\n")

```

- **全字段校验**：对模板名称、应用模板、执行脚本以及交互状态进行非空校验。
- **唯一性冲突检查**：通过 `model_name` 查重。如果库里已经有一个同名模板，则拦截请求，防止数据覆盖或混淆。
- **文本预处理（关键）**：
 - **换行符清洗**：使用 `.replace("\r\n", "\n")` 处理脚本内容。这是为了解决 Windows 环境编辑的脚本上传到 Linux 集群运行时，可能因回车符导致的脚本报错（即常见的 `^M` 问题）。

```

147         data = {
148             "source": user,
149             "model_name": model_name,
150             "param_data": param_data,
151             "script_exec": script_exec,
152             "is_remote": is_remote,
153             "create_time": ctime
154         }
155
156         if script_remote is not None:
157             script_remote = script_remote.replace("\r\n", "\n")
158             data["script_remote"] = script_remote
159
160         TbAppModel.objects.create(**data)
161         result = {
162             "code": 0,
163             "message": "success"
164         }
165
166         return result

```

- **动态参数赋值：**根据是否传入 `script_remote` 来动态构建存储字典。

3. 业务场景复盘

- **场景：**管理员需要新增一个“深度学习训练模板”。
- **流程：**在界面输入脚本逻辑，提交后后端通过此接口将其保存。之后其他用户就可以基于这个“模子”去快速创建具体的作业。

三、DELETE 接口：删除模型模板

1. 核心功能：清理冗余配置

该接口的作用是：从系统中永久移除不再需要的应用模型。

2. 后端核心逻辑拆解


```

168     def delete(self, request):
169         model_id = request.data.get("model_id", None)
170
171         result = {
172             "code": -1,
173             "message": "failed"
174         }
175         if model_id in ["", None]:
176             result["err"] = "请指定模板ID"
177             return result
178
179         obj = TbAppModel.objects.filter(app_model_id=model_id).exclude(source="plat")
180         if obj.count() == 0:
181             result["err"] = "模板不存在"
182             return result
183
184         obj.delete()
185         result = {
186             "code": 0,
187             "message": "success"
188         }
189
190         return result

```

• 权限/来源保护（核心逻辑）：

- 代码逻辑：`.exclude(source="plat")`。
- 目的：这是一种**硬保护机制**。系统自带的“平台级（plat）”内置模板是不允许被删除的。只有用户自己创建的模板才会被该接口执行 `obj.delete()`。
- **逻辑自治**：先过滤 ID，再排除平台模板，最后判断剩余对象数量。如果对象被排除后计数为 0，则提示“模板不存在”（实际上可能是因为该模板是平台保护的）。

3. 业务场景复盘

- **场景**：管理员清理自己测试时创建的废弃脚本。
- **流程**：勾选模板并删除。如果管理员试图删除系统自带的“标准 HPL 测试模板”，由于 `source="plat"` 的存在，操作会被后端拦截，确保系统基础功能不崩盘。

四、PUT 接口：更新模板基础信息

1. 核心功能：模板名称修正

从代码看，该 PUT 接口目前侧重于修改模板的**名称**。

2. 后端核心逻辑拆解

```

192     def put(self, request):
193         model_id = request.POST.get("model_id", None)
194         model_name = request.POST.get("model_name", None)
195
196         result = {
197             "code": -1,
198             "message": "failed"
199         }
200         if model_id in ["", None]:
201             result["err"] = "请指定模板ID"
202             return result
203         if model_name in ["", None]:
204             result["err"] = "请指定模板名称"
205             return result
206
207         obj = TbAppModel.objects.filter(app_model_id=model_id).exclude(source="plat")
208         if obj.count() == 0:
209             result["err"] = "模板不存在"
210             return result

```

- **保护逻辑延续：**同样使用了 `.exclude(source="plat")`，意味着平台内置的模板不仅不能删，名称也不能改，确保了系统的规范性。

```

212         obj.exclude(model_name=model_name).update(model_name=model_name)
213         result = {
214             "code": 0,
215             "message": "success"
216         }
217
218         return result

```

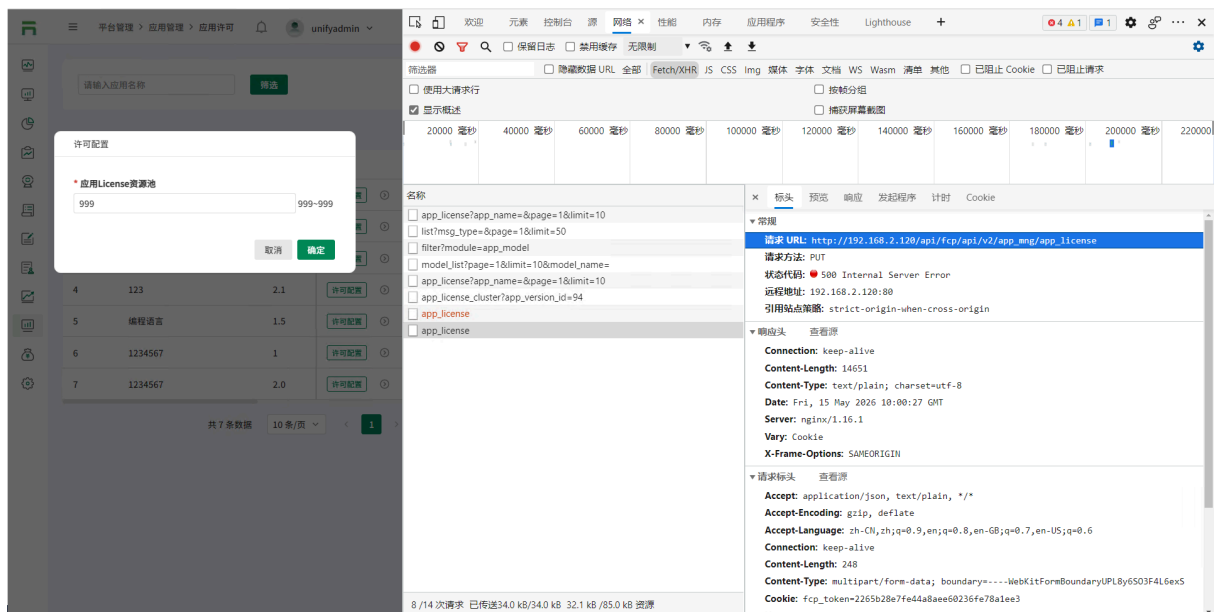
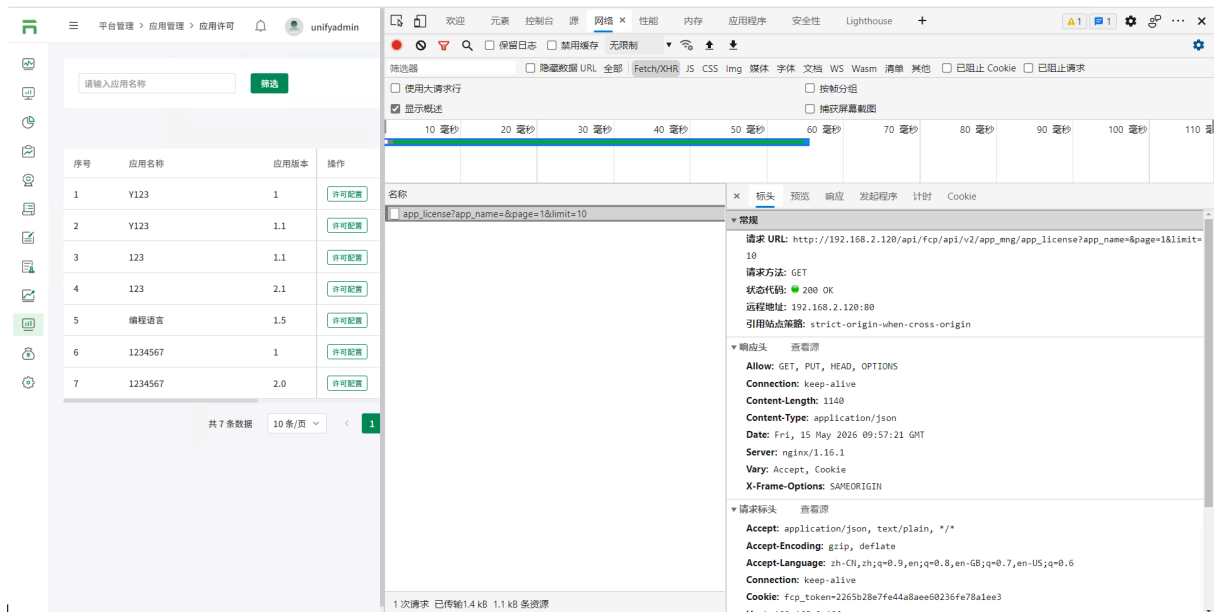
- **原子化更新：**使用 `obj.exclude(model_name=model_name).update(...)` 进行更新。
 - **优化点：**这里加了一个 `exclude` 过滤条件。如果新名字和旧名字一模一样，则不触发实际的数据库写操作，节省了性能。

3. 业务场景复盘

- **场景：**管理员觉得之前起的“测试脚本 1”太随意了，想改成“2026 第一季度 GPU 压力测试脚本”。
- **流程：**在页面上点击重命名，调用此接口完成数据库字段更新。

第四类：应用许可

一 应用许可接口



views.py

```

262 class AppLicenseView(APIView):
263     @method_decorator(permission(usertype=["admin"], permission="plat_set_get"))
264     def get(self, request):
265         # TODO 部分假数据, 后续继续完善 (完成)
266         r = AppLicense().get(request)
267         return Response(r)
268
269     @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
270     def put(self, request):
271         r, event = AppLicense().put(request)
272         # 记录日志
273         if r["code"] == 0 and event is not None:
274             username = TokenOp().get_username(request)
275             LogWrite().action_log_write(username=username,
276                                         operate_type="配置应用许可",
277                                         operate_content=event, state=True)
278         else:
279             r["message"] = r["err"]
280         return Response(r)

```

一、GET 接口：应用许可列表查询

1. 核心功能：资源池状态监控

该接口的作用是：提供一个全局视图，展示所有外部应用的 License 使用情况。它不仅返回基本信息，还实时计算出“总量、已用、剩余、已分配”四个关键指标，帮助管理员监控软件授权是否耗尽。

```

class AppLicense(object):
    def get(self, request):
        category_name = request.GET.get("app_name", None)
        page_index = request.GET.get("page", 1)
        page_size = request.GET.get("limit", 10)

        if page_index in ["", None]:
            page_index = 1
        if page_size in ["", None]:
            page_size = 10

        page_index = int(page_index)
        page_size = int(page_size)
        if page_index < 1:
            page_index = 1
        if page_size < 1:
            page_size = 10

```

2. 后端核心逻辑拆解

```

39
40         c_obj = TbAppCategory.objects.filter(is_extern=1).exclude(pid=0)
41         if category_name not in ["", None]:
42             c_obj = c_obj.filter(category_name__contains=category_name)
43
44         category_id_list = c_obj.values_list("app_category_id", flat=True)
45
46         obj = TbAppVersion.objects.filter(is_extern=1, category_id__in=category_id_list)
47         val = obj.values("app_version_id", "category_id", "version")
48

```

- 多级数据穿透：

- 从 `TbAppCategory` 筛选外部应用分类。
- 通过分类 ID 找到 `TbAppVersion` 中的具体版本。
- 最后关联 `TbAppLicense` 获取对应的 License 配置。

```

49         total_num = len(val)
50
51         page_num = int(total_num / page_size)
52         else_num = total_num % page_size
53         if else_num > 0:
54             page_num = page_num + 1
55
56         if page_index > page_num:
57             page_index = page_num
58
59         d_i_s = page_size * (page_index - 1)
60         d_i_e = page_size * page_index
61
62         if d_i_s < 0:
63             d_i_s = 0
64
65         val = val[d_i_s:d_i_e]

```

- 严谨的分页处理：手动计算了 `page_num`（总页数），并处理了 `page_index` 超出范围的边界情况。

```

70         for i in val:
71             category_id = i["category_id"]
72             app_category_name = c_obj.filter(app_category_id=category_id).values("category_name")[0]["category_name"]
73
74             app_version_id = i["app_version_id"]
75             version = i["version"]
76
77             l_obj = TbAppLicense.objects.filter(is_bind=1, app_version_id=app_version_id)
78             l_val = l_obj.values("app_license_id", "lic_num", "lic_name_map", "lic_name")
79
80             app_license_id = None
81             lic_num = 0
82             lic_num_use = 0
83             lic_num_alloc = 0
84
85             if len(l_val) > 0:
86                 app_license_id = l_val[0]["app_license_id"]
87                 lic_num = int(l_val[0]["lic_num"])
88
89                 # lic_name_map = l_val[0]["lic_name_map"]
90                 lic_name = l_val[0]["lic_name"]
91
92                 lic_num_use = 0
93                 jv_obj_f = jv_obj.filter(license_name=lic_name)
94                 jv_val_f = jv_obj_f.values("license_num")
95                 for k in jv_val_f:
96                     lic_num_use = lic_num_use + int(k["license_num"])
97
98                 l_c_obj = TbAppLicenseCluster.objects.filter(app_license_id=app_license_id)
99                 lic_num_alloc = l_c_obj.aggregate(total=Sum("lic_num"))["total"]
100
101                 lic_num_free = lic_num - lic_num_use

```

• 实时动态计算（核心指标）：

- **lic_num_use (实时使用数)**：通过查询 `TbJobView` 中所有 `completed_status=0`（未完成）的任务，累加这些任务当前占用的 License 数量。
- **lic_num_alloc (已分配数)**：查询 `TbAppLicenseCluster` 表，统计该 License 已经预分配给各个集群的总额度。
- **lic_num_free (剩余可用数)**：计算逻辑为 `总数 - 实时使用数`。

```

103     ..... i_d = {
104     .....         "s_category_name": app_category_name,
105     .....         "app_version_id": app_version_id,
106     .....         "version": version,
107     .....         "app_license_id": app_license_id,
108     .....         "lic_num": lic_num,
109     .....         "lic_num_use": lic_num_use,
110     .....         "lic_num_free": lic_num_free,
111     .....         "lic_num_alloc": lic_num_alloc
112     .....     }
113
114     ..... data["s_category"].append(i_d)
115
116     ..... data["count"] = total_num
117     ..... result = {
118     .....         "code": 0,
119     .....         "message": "success",
120     .....         "data": data
121     .....     }
122
123     ..... return result

```

3. 业务场景复盘

- **管理员视角：**打开“应用许可”管理页面。
- **数据呈现：**页面展示“MATLAB v2024b，总授权 100，当前正在跑的任务占用了 20 个，剩余 80 个可用，其中 50 个已经分配给了‘计算集群 A’”。

二、PUT 接口：应用许可配额修改

1. 核心功能：许可资源扩容/缩容

该接口的作用是：允许管理员手动调整某个应用版本的 License 总量（即修改资源池大小）。

```

125     ... def put(self, request):
126         ...     app_license_id = request.POST.get("app_license_id", None)
127         ...     lic_num = request.POST.get("lic_num_all", None)
128
129         ...     event = None
130         ...     result = {
131             ...         "code": -1,
132             ...         "message": "failed"
133         ...     }
134         ...     if app_license_id in ["", None]:
135             ...         result["err"] = "请指定应用许可id"
136             ...         return result, event
137         ...     if lic_num in ["", None]:
138             ...         result["err"] = "请指定许可数量"
139             ...         return result, event
140
141         ...     obj = TbAppLicense.objects.filter(app_license_id=app_license_id)
142         ...     val = obj.values("lic_num", "app_version_id", "lic_name_map")
143         ...     if len(val) == 0:
144             ...         result["err"] = "应用许可不存在"
145             ...         return result, event

```

2. 后端核心逻辑拆解

```

146
147     ...     l_c_obj = TbAppLicenseCluster.objects.filter(app_license_id=app_license_id)
148     ...     lic_num_alloc = l_c_obj.aggregate(total=Sum("lic_num"))["total"]
149     ...     if int(lic_num) < int(lic_num_alloc):
150         ...         result["err"] = "数量需大于已分配数量"
151         ...         return result, event
152

```

• 防爆护栏（逻辑校验）：

- 代码逻辑： `if int(lic_num) < int(lic_num_alloc)`。
- 关键约束：新设置的 License 总量不允许小于已经分配给各个集群的数量总和。这防止了“超卖”导致的逻辑冲突。


```

154         ori_lic_num = val[0]["lic_num"]
155         if str(lic_num) != str(ori_lic_num):
156             ctime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
157             obj.update(lic_num=lic_num, update_time=ctime)
158
159         # 日志
160         from common.config.project import LOG_PREFIX
161         version = None
162         category_name = None
163
164         app_version_id = val[0]["app_version_id"]
165         v_obj = TbAppVersion.objects.filter(is_extern=1, app_version_id=app_version_id)
166         v_val = v_obj.values("category_id", "version")
167         if len(v_val) > 0:
168             category_id = v_val[0]["category_id"]
169             version = v_val[0]["version"]
170
171         c_obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)
172         if c_obj.count() > 0:
173             category_name = c_obj.values("category_name")[0]["category_name"]

```

• 差异化更新与审计：

- 只有当新值与旧值不同（`str(lic_num) != str(ori_lic_num)`）时才触发更新，节省数据库开销。
- 富文本日志（Event 生成）：**代码拼接了一段带有 HTML `<span>` 标签的详细日志。它不仅记录了修改操作，还通过 `changed_val` CSS 类名标记了变化，方便前端在操作日志中高亮显示。

```

175         #
176         # op_obj = "应用名称: %s; 应用版本: %s" % (category_name, version)
177         # log_bef = "%s" % (LOG_PREFIX["bef"], ori_lic_num)
178         # log_aft = "%s" % (LOG_PREFIX["aft"], lic_num)
179         #
180         # event = "%s\n%s\n%s" % (op_obj, log_bef, log_aft)
181
182         op_obj = '<p class="log log_obj"><span class="obj">应用名称: </span><span class="val">%s</span></p>' % (category_name, version)
183         log_bef = '<p class="log log_content bef"><span class="bef">%s</span><span class="val">%s</span></p>' \
184             % (LOG_PREFIX["bef"], ori_lic_num)
185
186         lic_num_css = "changed_val"
187         log_aft = '<p class="log log_content aft"><span class="aft">%s</span><span class="val">%s</span></p>' \
188             % (LOG_PREFIX["aft"], lic_num_css, lic_num)
189
190         event = "%s%s" % (op_obj, log_bef, log_aft)
191
192         result = {
193             "code": 0,
194             "message": "success"
195         }
196
197         return result, event

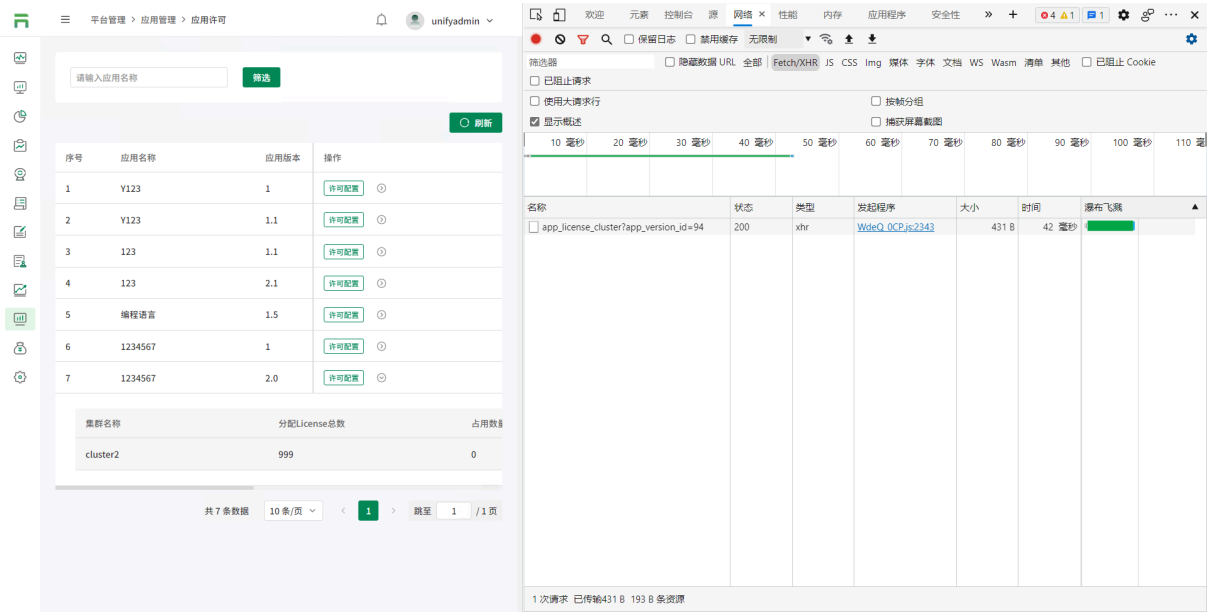
```

3. 业务场景复盘

- 扩容操作：**公司新采购了 50 个 License，管理员将“999”填入对话框（如你截图 `image_071de0.png` 所示）。

- **接口反馈：**后端校验成功后更新数据库，并在系统操作日志中留下记录：“应用名称：XXX，将许可数量由 100 修改为 999”。

二 应用许可节点接口



views.py

```
class AppLicenseClusterView(APIView):
    @method_decorator(permission(usertype=["admin"], permission="plat_set_get"))
    def get(self, request):
        # TODO 部分假数据，后续继续完善（完成）
        r = AppLicenseCluster().get(request)
        return Response(r)

    @method_decorator(permission(usertype=["admin"], permission="plat_set_op"))
    def put(self, request):
        # TODO 限制集群许可实际数量，后续继续完善（完成）
        r, event = AppLicenseCluster().put(request)
        # 记录日志
        if r["code"] == 0 and event is not None:
            username = TokenOp().get_username(request)
            LogWrite().action_log_write(username=username,
                                         operate_type="应用许可-集群限制",
                                         operate_content=event, state=True)
        return Response(r)
```

一、GET 接口：获取集群许可分配详情

1. 核心功能：分集群配额监控

该接口的作用是：针对某个特定的应用版本，查询它在各个集群上的分配情况。它能清晰地展示：总共多少许可、已经分给每个集群多少、每个集群当前实际被占用了多少。

```
13 class ApplicenseCluster(object):
14     def get(self, request):
15         app_version_id = request.GET.get("app_version_id", None)
16
17         result = {
18             "code": -1,
19             "message": "failed"
20         }
21         if app_version_id in ["", None]:
22             result["err"] = "请指定应用版本id"
23             return result
24
25         l_obj = TbAppLicense.objects.filter(app_version_id=app_version_id)
26         # app_license_id_list = l_obj.values_list("app_license_id", flat=True)
27         if l_obj.count() == 0:
28             result["err"] = "应用许可不存在"
29             return result
30         l_val = l_obj.values("app_license_id", "lic_name", "lic_num")
31         app_license_id = l_val[0]["app_license_id"]
32         lic_name = l_val[0]["lic_name"]
33         t_lic_num = l_val[0]["lic_num"]
34
35         remain_alloc = int(t_lic_num)
```

2. 后端核心逻辑拆解

```
37         # lc_obj = TbAppLicenseCluster.objects.filter(app_license_id__in=app_license_id_list)
38         lc_obj = TbAppLicenseCluster.objects.filter(app_license_id=app_license_id)
39         lc_val = lc_obj.values("cluster_id", "lic_num")
40
```

• 第一步：全局配额对齐

- 根据 `app_version_id` 锁定 `TbAppLicense` 表，获取该应用的总许可数 (`t_lic_num`)。

• 第二步：提取已分配数据

- 从 `TbAppLicenseCluster` 表中提取出目前已经分配到各个集群的记录 (`lc_val`)。

• 第三步：实时占用计算（双层循环）

```

41         # job_view数据表查询未结束的作业信息
42         jv_obj = TbJobView.objects.filter(completed_status=0, license_name=lic_name,
43         .....job_state__in=JOB_STATE_LIC_LIST)
44         lc_data = {}
45         cluster_id_list = []
46         for i in lc_val:
47             cluster_id = i["cluster_id"]
48             lic_num = int(i["lic_num"])
49
50             lic_use = 0
51             jv_obj_c = jv_obj.filter(cluster_id=cluster_id)
52             jv_val_c = jv_obj_c.values("license_num")
53             for k in jv_val_c:
54                 lic_use = lic_use + int(k["license_num"])
55
56             lc_data[cluster_id] = {
57                 "lic_num": lic_num,
58                 "lic_use": lic_use,
59                 "lic_free": lic_num - lic_use
60             }
61
62         cluster_id_list.append(cluster_id)

```

- 程序遍历每一个已分配的集群，并去 `TbJobView` 表里“搜身”：找出当前在这个集群上运行（`completed_status=0`）且使用了该 License 的所有作业。
- 计算指标：
 - `lic_num`：该集群分到的“死配额”。
 - `lic_use`：该集群目前被作业“活占用”的数量。
 - `lic_free`：该集群内部剩余可用的额度。
- 第四步：剩余池计算
 - 通过 `remain_alloc = total - sum(allocated)`，算出总仓库里还有多少许可处于“待分配”状态，并将其标记在返回结果中。

```

64         remain_alloc = remain_alloc - lic_num
65
66         # cluster_id_list = lc_obj.values_list("cluster_id", flat=True)
67
68         c_obj = TbCluster.objects.filter(cluster_id__in=cluster_id_list)
69         c_val = c_obj.values("cluster_id", "name")
70
71         c_data = []
72         for k in c_val:
73             cluster_id = k["cluster_id"]
74             name = k["name"]
75
76             c_d = {
77                 "cluster_id": cluster_id,
78                 "cluster_name": name,
79                 "cluster_lic_num": lc_data[cluster_id]["lic_num"],
80                 "cluster_lic_num_use": lc_data[cluster_id]["lic_use"],
81                 "cluster_lic_num_free": lc_data[cluster_id]["lic_free"],
82                 "cluster_lic_num_alloc": remain_alloc
83             }
84
85             c_data.append(c_d)
86
87         result = {
88             "code": 0,
89             "message": "success",
90             "data": {"cluster": c_data}
91         }
92
93         return result

```

3. 业务场景复盘

- **场景：**管理员点击左侧表格中的“展开”或“详情”图标。
- **展示：**下方刷出一个子表格（如截图底部所示），显示：`cluster2` 分配了 999 个 License，目前占用 0 个。

二、PUT 接口：调整集群许可分配配额

1. 核心功能：跨集群资源调配（含底层同步）

该接口的作用是：手动修改某个集群分到的 License 数量。它是这套系统中最重的操作之一，因为它不仅改数据库，还要直接调用 HPC 底层调度器（如 Slurm）的命令。

```

95     def put(self, request):
96         app_license_id = request.POST.get("app_license_id", None)
97         cluster_id = request.POST.get("cluster_id", None)
98         lic_num = request.POST.get("cluster_lic_num", None)
99
100         event = None
101         result = {
102             "code": -1,
103             "message": "failed"
104         }
105         if app_license_id in ["", None]:
106             result["err"] = "请指定应用许可id"
107             return result, event
108         if cluster_id in ["", None]:
109             result["err"] = "请指定集群id"
110             return result, event
111         if lic_num in ["", None]:
112             result["err"] = "请指定分配集群许可数量"
113             return result, event
114
115         l_obj = TbApplicense.objects.filter(app_license_id=app_license_id)
116         if l_obj.count() == 0:
117             result["err"] = "应用许可不存在"
118             return result, event
119
120         lic_num_total = l_obj.values("lic_num")[0]["lic_num"]
121
122         l_c_obj = TbApplicenseCluster.objects.filter(app_license_id=app_license_id)
123         obj = l_c_obj.filter(cluster_id=cluster_id)
124         if obj.count() == 0:
125             result["err"] = "应用许可集群信息不存在"
126             return result, event

```

2. 后端核心逻辑拆解

• 第一步：溢出校验（仓库检查）

```

128         lic_num_alloc = l_c_obj.aggregate(total=Sum("lic_num"))["total"]
129         remain_lic_num = int(lic_num_total) - int(lic_num_alloc)
130
131         ori_lic_num = obj.values("lic_num")[0]["lic_num"]
132         if int(lic_num) - int(ori_lic_num) > remain_lic_num:
133             result["err"] = "超出可分配数量"
134             return result, event
135

```

- 后端会计算： $(\text{总许可数} - \text{已分配给其他集群的总数})$ 。如果管理员填写的数字超过了这个剩余值，则报错“超出可分配数量”。
- 第二步：底层调度器同步（核心硬核逻辑）

```

136         # 判断集群是否存在许可信息
137         lic_name_map = l_obj.values("lic_name_map")[0]["lic_name_map"]
138         cmd = f"scontrol show license {lic_name_map}@{LIC_SERVER} -o"
139
140         ret, out = Base().cluster_cmd(cluster_id=cluster_id, cmd=cmd)
141         out = out.strip()
142         if ret != 0 or len(out) == 0:
143             result["err"] = "集群未配置指定许可"
144             return result, event
145
146         #
147         cmd_c = f"sacctmgr update resource name={lic_name_map}. " \
148             f"server={LIC_SERVER} set count={lic_num} -i"
149         ret_c, out_c = Base().cluster_cmd(cluster_id=cluster_id, cmd=cmd_c)
150         if ret_c != 0:
151             result["err"] = "设置集群许可数量失败"
152             result["err2"] = out_c.strip().split("\n")
153         return result

```

- 代码通过 `Base().cluster_cmd` 发送了两个关键的 Shell 命令：
 1. **查询检测**: `scontrol show license...`。先去集群上看看底层调度器是否认这个 License。
 2. **强制更新**: `sacctmgr update resource... set count={lic_num}`。
- **重要性**: 这确保了“账面数据”与“运行环境”一致。只有底层 `sacctmgr` 修改成功了，调度器才会真正限制该集群上的作业运行数量。
- **第三步：数据库与审计同步**

```

155         if str(lic_num) != str(ori_lic_num):
156             obj.update(lic_num=lic_num)
157
158         # 日志
159         cluster_name = None
160         c_obj = TbCluster.objects.filter(cluster_id=cluster_id)
161         if c_obj.count() > 0:
162             cluster_name = c_obj.values("name")[0]["name"]
163
164         from dbtable.app import AppVersion as TbAppVersion, AppCategory as TbAppCategory
165         from common.config.project import LOG_PREFIX
166
167         app_version_id = l_obj.values_list("app_version_id", flat=True)[0]
168         version = None
169         category_name = None
170         v_obj = TbAppVersion.objects.filter(is_extern=1, app_version_id=app_version_id)
171         if v_obj.count() > 0:
172             v_val = v_obj.values("category_id", "version")
173             category_id = v_val[0]["category_id"]
174             version = v_val[0]["version"]
175
176         c_obj = TbAppCategory.objects.filter(is_extern=1, app_category_id=category_id)
177         if c_obj.count() > 0:
178             category_name = c_obj.values("category_name")[0]["category_name"]
179
180         #
181         op_obj = "应用名称: %s; 应用版本: %s; 集群名称: %s" % (category_name, version, cluster_name)
182         log_bef = "%s分配License总数: %s" % (LOG_PREFIX["bef"], ori_lic_num)
183         log_aft = "%s分配License总数: %s" % (LOG_PREFIX["aft"], lic_num)
184         #
185         event = "%s\n%s\n%s" % (op_obj, log_bef, log_aft)
186

```

- 底层命令执行成功后，更新 `TbAppLicenseCluster` 表。

```

187
188     op_obj = '''<p class="log_log_obj"><span class="obj">应用名称:
189     </span><span class="val">%s</span></span><span class="obj">应用版本:
190     </span><span class="val">%s</span></span><span class="obj">集群名称: </span><span class="val">%s</span></p>''' \
191         % (category_name, version, cluster_name)
192
193     log_bef = '''<p class="log_log_content bef"><span class="bef">%s</span><span class="obj">分配License总数: </span><span class="val">%s</span></p>''' \
194         % (LOG_PREFIX["bef"], ori_lic_num)
195
196     lic_num_css = "changed_val"
197     log_aft = '''<p class="log_log_content aft"><span class="aft">%s</span><span class="obj">分配License总数: </span><span class="val">%s</span></p>''' \
198         % (LOG_PREFIX["aft"], lic_num_css, lic_num)
199
200     event = "%s%s" % (op_obj, log_bef, log_aft)
201
202     result = {
203         "code": 0,
204         "message": "success"
205     }
206     return result, event
207

```

- 富文本日志记录：**生成详细的日志，记录了应用名、版本、集群名，以及 License 从 `X` 变动到 `Y` 的过程，并标记为 `changed_val` 高亮展示。

3. 业务场景复盘

- 操作：**管理员发现 `cluster2` 的计算压力很大，原本分给它的 100 个 License 不够用了，于是从总池子里再拨 200 个给它。
- 效果：**提交后，后端自动去 `cluster2` 的调度节点执行了 `sacctmgr` 更新命令。之后，该集群上的用户就能提交更多需要该 License 的作业了。